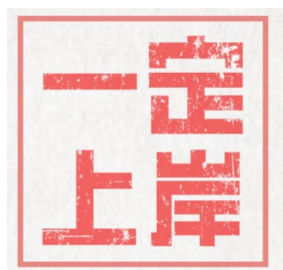


26 考研 4081000 题

relax 团队



ds 做题本

一 绪论

1. 以下关于数据结构的说法中，正确的是（ ）。
 - A. 数据的逻辑结构独立于其存储结构
 - B. 数据的存储结构独立于其逻辑结构
 - C. 数据的逻辑结构唯一决定其存储结构
 - D. 数据结构仅由其逻辑结构和存储结构决定
2. 设有如下遗产继承规则:丈夫和妻子可以互相继承遗产，子女可以继承父亲和母亲的遗产，子女间不能相互继承，则表示该遗产继承关系最合适的数据结构应该是（ ）
 - A. 树
 - B. 图
 - C. 线性表
 - D. 集合
3. 下列关于数据结构的说法，正确的有（ ）
 - I. 数据项是数据的基本单位
 - II. 抽象数据结构可以定义一个完整的数据结构
 - III. 循环队列属于数据结构中的物理结构
 - IV. 线索二叉树属于数据结构中的物理结构
 - A. II、III、IV
 - B. I、II、IV
 - C. I、III
 - D. 全部正确
4. 通常要求同一逻辑结构中的所有数据元素具有相同的特性，这意味着()
 - A. 数据具有同一特点
 - B. 不仅数据元素所包含的数据项的个数要相同，而且对应数据项的类型要一致
 - C. 每个数据元素都一样
 - D. 数据元素所包含的数据项的个数要相等
5. 某算法的时间复杂度为 $O(n^2)$ ，表明该算法的()。

- A. 问题规模是 n^2
- B. 执行时间等于 n^2
- C. 执行时间与 n^2 成正比
- D. 问题规模与 n^2 成正比

6. 下列逻辑结构属于线性结构的是 ()

- A. 树
- B. 图
- C. 集合
- D. 队列

7. 下列不属于物理结构的是 ()

- A. 顺序表
- B. 单链表
- C. 散列表
- D. 有序表

8. 数据结构和数据类型的形式定义分别为:

Data-Structure=(D, R)

Data-Type=(D, R, P)

试选择 D、R、P 的确切含义是 ()

- A. 数据, 数据元素, 存储结构
- B. 数据元素, 数据关系, 基本操作
- C. 数据对象, 存储结构, 基本操作
- D. 数据对象, 数据关系, 基本操作

9. 下列代码的时间复杂度是 ()

```

C++
int x=90, y=100;
while(y>0){
    if(x>100){
        x=x-10;
        y--;
    }
    else x++;
}

```

- A. $O(n)$
- B. $O(n^2)$
- C. $O(n^3)$
- D. $O(1)$

10. 下列代码的时间复杂度是 ()

```

C++
for (int i = 1; i <= n; i ++ ) {
    for (int j = 1; j <= n; j ++ ) {
        k = 1;
        while (k <= n) k *= 5;
    }
}

```

- A. $O(n^2)$
- B. $O(n^3)$
- C. $O(n^2 \log_2 n)$
- D. $O(n \log_2 n)$

11. 下列代码的时间复杂度是 ()

```

C++
int cnt = 1, sum = 0;
while(sum <= n){
    sum += cnt;
    cnt ++ ;
}

```

- A. $O(n)$

- B. $O(n^2)$
- C. $O(\sqrt{n})$
- D. $O(1)$

12. 假定 $n > 2$, 计算机执行下面的语句时, 语句 S 的执行次数为 ()。

```
C++
for (i = 1; i < n - 1; i ++ )
    for (j = n; j >= i; j --)
        S;
```

- A. $n(n-1)$
- B. $(n+3)(n-2)/2$
- C. $(n+2)(n-1)/2$
- D. $n(n-1)/2$

13. 以下程序的时间复杂度是()

```
C++
for (int i = 1; i <= n; i ++ ) {
    for (int j = 2 * i; j <= n; j ++ ){
        y += i * j;
    }
}
```

- A. $O(n)$
- B. $O(n^2)$
- C. $O(n \log_2 n)$
- D. $O(\log_2 n)$

14. 下列代码的时间复杂度是 ()

```
C++
for(i=1;i<=n;i++)
    for(j=1;j<=n;j++){
        c[i][j]=0;
        for(k=1;k<=n;k++)
            c[i][j]=c[i][j]+a[i][k]*b[k][j];
    }
```

```
}
```

- A. $O(n)$
- B. $O(n^2)$
- C. $O(n^2 \log_2 n)$
- D. $O(n^3)$

15. 下列代码的时间复杂度为 ()

```
C++
for (int i = 1; i <= n; i ++ )
    for (int j = 1; j <= i; j *= 2)
        for (int k = 1; k <= j; k ++ )
            count ++ ;
```

- A. $O(n^2 \log_2 n)$
- B. $O(n^2)$
- C. $O(n \log_2 n)$
- D. $O(n^3)$

16. 以下代码的时间复杂度为 ()

```
C++
void F(int n) {
    int sum = 0;
    for (int i = 1; i <= n; i ++ ) {
        for (int j = 0; j < n; j += i) {
            sum += j;
        }
    }
}
```

- A. $O(n^2)$ B. $O(n^{\frac{3}{2}})$ C. $O(n \log_2 n)$ D. $O(n^3)$

17. 下面程序的时间复杂度是 ()

```
java
void func(int n){
```

```

int i, j, k;
int count = 0;
for (int i = 1; i <= n; i *= 2){
    for (int j = 1; j <= i; j ++ ){
        for (k = 1; k <= j; k ++ ){
            count ++ ;
        }
    }
}
}

```

- A. $O(n^3)$
- B. $O(n^2)$
- C. $O(n \log_2 n)$
- D. $O(n^2 \log_2 n)$

18. 对于下列求 Fibonacci 数的算法，其时间复杂度为 ()

```

C++
int Fibonacci(int n) {
    if(n == 1 || n==2) return 1;
    return Fibonacci(n-1) + Fibonacci(n-2);
}

```

- A. $O(n)$
- B. $O(n^2)$
- C. $O(\sqrt{n})$
- D. 指数复杂度

19. 以下是经典汉诺塔问题的伪代码，在调用 `hanoi(n, 'A', 'C', 'B');` 时，时间复杂度是 ()。

```

Java
void hanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 1) {
        只有一个盘子时，直接移动到目标柱子
        return;
    }
    hanoi(n-1, from_rod, aux_rod, to_rod);
    将最大的盘子（第 n 个）从起始柱子移动到目标柱子
    hanoi(n-1, aux_rod, to_rod, from_rod);
}

```

```
}
```

- A. $O(2^n)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n \log n)$

20. 设二叉树共有 n 个结点，下列程序段的时间复杂度是 ()。

```
SQL
int maxFunc(TreeNode root) {
    if (root == NULL) return 0;
    return max(maxFunc(root->left), maxFunc(root->right)) + 1;
}
```

- A. $O(\log_2 n)$
- B. $O(n)$
- C. $O(n \log_2 n)$
- D. $O(2^n)$

21. 下列代码的时间复杂度是 ()

```
go
int func(int n) {
    if (n == 1) return 1;
    return func(n - 1) * n;
}
```

- A. $O(n^2)$
- B. $O(n)$
- C. $O(2^n)$
- D. $O(n \log_2 n)$

22. 下列递归表达式的复杂度是多少()

$$T(n) = 3T(n/4) + \theta(n^2)$$

- A. $O(n^2)$
- B. $O(2^n)$
- C. $O(n)$
- D. $O(n^2 \log_2 n)$

23. 下列递归代码的时间复杂度为 ()

```
C++
void process(int n) {
    if (n <= 1) return;

    int sum = 0;
    for (int i = 0; i < n; ++i) sum += i;
    printf("%d\n", sum);
    for (int i = 0; i < 2; ++i)
        process(n / 2);
}
```

- A. $O(n \log_2 n)$
- B. $O(n^2)$
- C. $O(n^2 \log_2 n)$
- D. $O(\log_2 n)$

24. 以下代码在最坏情况下的时间复杂度为()

```
C++
for(k=n-1;k>=1;--k)
    for(t=1;t<k; ++t)
        if(A[t]> A[t+1])
            swap(A[t],A[t+1]); //将 A[t]和 A[t+1]对换
```

- A. $O(n)$
- B. $O(n \log_2 n)$
- C. $O(n^3)$
- D. $O(n^2)$

25. 下列程序段的空间复杂度是 ()。

```
go
int func(int m,int n){
    if(m==n)
        return 1;
    else{
        int mid=(m+n)/2;
        return func(m,mid)+func(mid+1,n);
    }
}
```

```
}
```

- A. $O(\log_2 n)$
- B. $O(n)$
- C. $O(n \log_2 n)$
- D. $O(2^n)$

26. 若某算法的空间复杂度为 $O(1)$,则表示该算法 ()。

- A. 不需要任何辅助空间
- B. 所需辅助空间大小与问题规模 n 无关
- C. 不需要任何空间
- D. 所需空间大小与问题规模 n 无关

二 线性表

1. 线性表是一个()。

- A.有限序列，可以为空
- B.有限序列，不能为空
- C.无限序列，可以为空
- D.无限序列，不能为空

2. ()是一个线性表

- A.由 n 个实数组成的集合
- B.由 100 个字符组成的序列
- C.由所有整数组成的序列
- D.邻接表

3. 关于线性表的顺序存储和链式存储，下列说法正确的是 ()

- A. 链式存储方式存储密度大于顺序存储方式
- B. 都支持随机访问
- C. 链式存储中，逻辑上相邻的元素在物理上不一定相邻
- D. 在 k 个位置进行插入，顺序存储比链式存储更方便

4. 使用一个顺序表(顺序存储结构)不能完整地表示()

- A.有向图
- B.树
- C.二叉树
- D.栈

5. 下列说法错误的是 ()

- I. 单链表是一种线性表，每个元素都有其前驱和后继
 - II. 单链表的存储密度比顺序表低
 - III. 需要经常修改线性表 L 中的结点值，用链表比较方便
 - IV. 静态链表删除或插入不需要移动元素
- A. III、IV B. I、II、III C. I、III D. I、II、III、IV

6. 下述说法不正确的是 ()

- A. 数组可看成线性结构的一种推广，因此与线性表一样，可以对它进行插入、删除等操作
- B. 稀疏矩阵以三元组表压缩存储后，必会失去随机存取功能
- C. 从逻辑结构上看，n 维数组的每个元素均属于 n 个向量
- D. 数组也能采用链式存储结构

7. 下列关于顺序表的叙述中，正确的是 ()。

- A. 顺序表可以利用一维数组表示，因此顺序表与一维数组在逻辑结构上是相同的
- B. 在顺序表中，逻辑上相邻的元素物理位置上不一定相邻
- C. 顺序表可以进行随机存取
- D. 在顺序表中，每个元素的类型不必相同

8. 在双链表的两个结点之间插入一个新结点，需要修改 () 个指针域

- A. 1
- B. 3
- C. 4
- D. 2

9. 与单链表相比，双链表的优点之一是 ()

- A. 插入、删除操作更方便
- B. 可以进行随机访问
- C. 可以省略表头指针或表尾指针

D.访问前后相邻结点更灵活

10. 一个链表最常用的操作是在末尾插入结点和删除结点，则选用()最节省时间。

A.带头结点的双循环链表

B.单循环链表

C.带尾指针的单循环链表

D.单链表

11. 设一个链表最常用的操作是在链尾之后插入新结点和删除链表的尾结点，则选() 效率最高。

A. 单链表

B. 循环单链表

C. 带链尾指针的循环单链表

D. 带头结点的循环双链表

12. 对于没有尾指针的单链表，将 n 个元素采用头插法建立单链表的时间复杂度为()，采用尾插法建立单链表的时间复杂度为()。

A. $O(1)$

B. $O(n\log_2 n)$

C. $O(n)$

D. $O(n^2)$

13. 有一个单向链表，头指针和尾指针分别为 p, q ，以下()操作的复杂度会受链表长度的影响

A 删除头部元素

B 删除尾部元素

C 头部元素之前插入一个元素

D 尾部元素之后插入一个元素

14. 设指针变量 p 指向单链表中结点 A ，若删除单链表中结点 A ，则需要修改指针的

操作序列为 ()

A $q=p \rightarrow \text{next}; p \rightarrow \text{data}=q \rightarrow \text{data}; p \rightarrow \text{next}=q \rightarrow \text{next}; \text{free}(q);$

B $q=p \rightarrow \text{next}; q \rightarrow \text{data}=p \rightarrow \text{data}; p \rightarrow \text{next}=q \rightarrow \text{next}; \text{free}(q);$

C $q=p \rightarrow \text{next}; p \rightarrow \text{next}=q \rightarrow \text{next}; \text{free}(q);$

D $q=p \rightarrow \text{next}; p \rightarrow \text{data}=q \rightarrow \text{data}; \text{free}(q);$

15. 不带头结点的单链表, 设头指针为 h , 指针 s 指向插入结点, 那么头插的操作应该是 ()

A. $h \rightarrow \text{next} = s \rightarrow \text{next};$

B. $s \rightarrow \text{next} = h \rightarrow \text{next}; h \rightarrow \text{next} = s;$

C. $s \rightarrow \text{next} = h; h = s;$

D. $h \rightarrow \text{next} = s; h = s;$

16. 设有线性表 $A=(a_1, a_2, \dots, a_m), B=(b_1, b_2, \dots, b_n)$ 。将线性表 A 、 B 交叉合并为线性表 $C=(a_1, b_1, a_2, b_2, \dots)$, 线性表 A 、 B 和 C 均以单链表结构存储, A 、 B 中长度更大的, 后续多出的元素全部放在 C 的最后, 那么时间复杂度为 ()

A. $O(m)$

B. $O(n)$

C. $O(m+n)$

D. 以上都错误

17. 设有一个长度为 n 的循环单链表, 若从表中删除首元结点的时间复杂度达到 $O(n)$, 则此时采用的循环单链表的结构可能是 ()。

A. 只有表头指针, 没有头结点

B. 只有表尾指针, 没有头结点

C. 只有表尾指针, 带头结点

D. 只有表头指针, 带头结点

18. 长度为 n 的单链表链接在长度为 m 的单链表 (都为不带头结点只有头指针的单链

表) 后面的时间复杂度为()

- A. $O(1)$
- B. $O(n)$
- C. $O(m)$
- D. $O(m+n)$

19. 已知头指针 h 指向一个带头结点的非空单链表, 结点结构为: $\{data, next\}$, 其中 $next$ 是指向后继结点的指针, p 指针指向该单循环链表中的一个结点, 在 p 指向的结点后插入一个由 s 指针指向的新结点, 正确操作是 ()

- A. $s \rightarrow next = p;$ $p \rightarrow next = s;$
- B. $p \rightarrow next = s;$ $s \rightarrow next = p \rightarrow next;$
- C. $s \rightarrow next = s;$ $p \rightarrow next = s;$
- D. $s \rightarrow next = p \rightarrow next;$ $p \rightarrow next = s;$

20. 对带头结点的循环单链表, 删除首元结点 (非最后一个结点), 需要修改 () 个指针

- A. 0
- B. 1
- C. 2
- D. 3

21. 设线性表中有 $2n$ 个元素, 算法 () 在单链表上实现要比在顺序表上实现效率更高。

- A. 删除所有值为 x 的元素
- B. 在最后一个元素的后面插入一个新元素
- C. 顺序输出前 K 个元素
- D. 交换第 i 个元素和第 $2n-i-1$ 个元素的值 ($i=0, 1, \dots, n-1$)

22. 下列说法正确的是()

- A. 分配给单链表的内存单元地址必须是连续的
- B. 与顺序表相比, 在链表中顺序访问所有节点, 其算法的效率比较低

C.从长度为 n 的顺序表中删除任何一个元素,时间复杂度都是 $O(1)$

D.向顺序表中插入一个元素,平均要移动大约一半的元素

23. 下列说法正确的是 ()

I. 不带头结点单链表中, 无论是插入还是删除元素, 都需要找到其前驱

II. 删除带头结点的循环双链表中某结点时, 只需要修改两个指针域

III. 带尾指针的单向循环链表中, 删除尾结点的时间复杂度是 $O(1)$

IV. 带头结点不带尾指针的单向循环链表, 在链头插入一个元素的时间复杂度是 $O(1)$

V. 链式存储结构都是动态申请空间, 各元素逻辑上相邻, 物理上不一定相邻

A. I、IV、V

B. II、IV

C. III、IV、V

D. I、II、V

24. 在双链表中 p 所指的结点之前插入一个结点 q 的操作为 ()

A. $p \rightarrow \text{prior} = 1$; $q \rightarrow \text{next} = p$; $p \rightarrow \text{prior} \rightarrow \text{next} = q$; $q \rightarrow \text{prior} = p \rightarrow \text{prior}$;

B. $q \rightarrow \text{prior} = p \rightarrow \text{prior}$; $p \rightarrow \text{prior} \rightarrow \text{next} = q$; $q \rightarrow \text{next} = p$; $p \rightarrow \text{prior} = q \rightarrow \text{next}$;

C. $q \rightarrow \text{next} = p$; $p \rightarrow \text{next} = q$; $q \rightarrow \text{prior} \rightarrow \text{next} = q$; $q \rightarrow \text{next} = p$;

D. $p \rightarrow \text{prior} \rightarrow \text{next} = q$; $q \rightarrow \text{next} = p$; $q \rightarrow \text{prior} = p \rightarrow \text{prior}$; $p \rightarrow \text{prior} = q$;

25. 已知带头结点的非空循环双链表, 指针 p 指向链表中非首且非尾的任意一个结点, 则执行下列语句序列实现的功能是 ()

```
C++
p->pre->pre->next = p;
p->next->pre = p->pre;
p->pre->next = p->next;
p->next = p->pre;
p->pre = p->pre->pre;
p->next->pre = p;
```

- A. 删除 p 指向的结点
- B. 将 p 指向的结点之前的结点插入 p 结点之后
- C. 将 p 指向的结点与其后继结点互换
- D. 将 p 指向的结点的后继与 p 指向的结点的前驱互换

26. 如果对线性表的运算只有 4 种，即删除第一个元素，删除最后一个元素，在第一个元素前面插入新元素，在最后一个元素的后面插入新元素则最好使用以下哪种存储结构？（ ）

- A 只有表尾指针没有表头指针的循环单链表。
- B 只有表尾指针没有表头指针的非循环双链表。
- C 只有表头指针没有表尾指针的循环双链表。
- D 既有表头指针也有表尾指针的循环单链表。

27. 下列操作可以在 $O(1)$ 的时间复杂度内完成的是（ ）。

- A. 删除单链表的第 n 个元素
- B. 在单链表的末尾添加一个元素
- C. 删除单向循环链表的头结点
- D. 删除双向循环链表的头结点

28. 设对 $n(n>1)$ 个元素的线性表的运算只有 4 种：①删除第一个元素；②删除最后一个元素；③在第一个元素之前插入新元素；④在最后一个元素之后插入新元素，则最好使用（ ）。

- A. 只有尾结点指针没有头结点指针的循环单链表
- B. 只有尾结点指针没有头结点指针的非循环双链表
- C. 只有头结点指针没有尾结点指针的循环双链表
- D. 既有头结点指针又有尾结点指针的循环单链表

29. 带头结点的双向循环链表 L 为空表的条件是（ ）

- A. $L \rightarrow next = L;$

- B. L = NULL;
- C. L->next->prior = NULL;
- D. L->prior = NULL;

30. 现有一块连续空间能用来保存数据，且常需要插入、删除数据，那么最适合使用()

- A . 单链表
- B . 静态链表
- C . 双链表
- D . 顺序表

31. 下列关于静态链表的说法中，正确的是()。

- I.静态链表兼具顺序表和单链表的优点，因此存取表中第 i 个元素的时间与 i 无关
- II.静态链表能容纳的最大元素个数在表定义时就确定了，以后不能增加
- III.静态链表与动态链表在元素的插入、删除上类似，不需要移动元素
- IV.相比动态链表，静态链表可能浪费较多的存储空间

- A. I、II、III
- B. II、III、IV
- C. I、III、IV
- D. I、II、IV

32. 已知表头元素为 c 的单链表在内存中的存储状态如下表所示：

下标	元素	链接地址
0	a	4
1	b	3
2	c	0
3	d	-1

4	e	1
---	---	---

删除元素 a 后，数组下标为 0, 2 的表项中地址内容为 ()

- A. 2, 4
- B. -1, 4
- C. -1, 3
- D. -2, 4

三、四 栈、队列、数组、字符串

1. 最适合用做链式队列的链表是 ()

- A. 带队首指针和队尾指针的循环单链表
- B. 带队首指针和队尾指针的非循环单链表
- C. 只带队首指针的非循环单链表
- D. 只带队首指针的循环单链表

2. 有一个空栈，栈顶指针指向栈顶元素（每个元素占一个存储单元），当前栈顶指针为 1006H，入栈方向为指针增大的方向。执行 Push、Push、Push、Push、Pop、Push、Pop、Push 操作后，栈顶指针的值为()。

- A. 1008H
- B. 1009H
- C. 1010H
- D. 100AH

3. 用字母“I”表示入栈，“O”表示出栈，对于入栈序列为“1234”，入栈出栈可交叉进行，得到出栈序列为“2431”的操作序列为 ()

- A. IIOIOIOO
- B. IIOIIOOO
- C. IIIOOOIO
- D. IOIIIOIOO

4. 有一个非空栈 S，栈顶指针 sp 指向栈顶元素（每个元素大小占一个存储单元），入栈方向是地址增大的方向，设入栈元素是 x，那么入栈操作是 ()

- A. $S[sp++] = x;$
- B. $S[++sp] = x;$
- C. $S[sp--] = x;$

D. $S[--sp] = x;$

5. 若元素“1, 2, ..., n”依次进入初始为空的栈中, 元素入栈后可停留可出栈, 直到所有的元素都出栈, 若第 k ($1 \leq k \leq n$) 个出栈元素是“1”, 那么第 $k+1$ 个出栈元素的可能取指有 () 种

- A. k
- B. $n - k$
- C. $n - 1$
- D. $k + 1$

6. 一个栈的入栈序列是 $1, 2, 3, \dots, n$, 则出栈序列 $P_1, P_2, P_3, \dots, P_n$ 中, 元素 1 的可能位置有多少个 ()

- A. 1
- B. $n-1$
- C. n
- D. $n/2$

7. 将 5 个字母“ooops”按顺序进栈, 则有 () 种不同的出栈顺序仍然可以得到“ooops”

- A. 1
- B. 3
- C. 5
- D. 6

8. 入栈序列为 1234 的出栈序列个数有 () 个

- A. 13
- B. 14
- C. 15
- D. 16

9. 一个栈(容量为 3)的进栈序列为 1,2,3,4, 那么有 () 个不同的出栈序列?

- A. 15
- B. 14
- C. 13
- D. 12

10. 设 n 个元素进栈序列是 $P_1, P_2, P_3, \dots, P_n$, 其输出序列是 $1, 2, 3, \dots, n$, 若 $P_n=1$, 则 $P_i (1 \leq i \leq n-1)$ 的值是 ()

- A. $n-i+1$
- B. $n-i$
- C. i
- D. 有多种可能

11. 若一个栈的入栈顺序为 1, 2, 3, 4, 5, 则不能得到的出栈顺序为()

- A. 4, 3, 2, 1, 5
- B. 3, 2, 5, 1, 4
- C. 4, 5, 3, 2, 1
- D. 3, 4, 2, 5, 1

12. 元素 a, b, c, d, e 依次进入初始为空的栈中, 若元素进栈后可停留、可出栈, 直到所有元素都出栈, 则在所有可能的出栈序列中, 以元素 d 开头的序列个数是()。

- A. 3
- B. 4
- C. 5
- D. 6

13. 采用共享栈的好处是()。

- A. 减少存取时间, 降低发生上溢的可能

- B.节省存储空间, 降低发生上溢的可能
- C.减少存取时间, 降低发生下溢的可能
- D.节省存储空间, 降低发生下溢的可能

14. 设有一个顺序共享栈 $share[0:n-1]$, 其中第一个栈顶指针 $top1$ 的初值为-1, 第二个栈顶指针 $top2$ 的初值为 n , 则判断共享栈满的条件是()。

- A. $top2-top1 == 1$
- B. $top1-top2 == 1$
- C. $top1==top2$
- D. $top1-top2 == 2$

15. 中缀表达式 $A * (B+C)/(D-E+F)$ 的后缀表达式是 ()

- A. $A*B+C/D-E+F$
- B. $ABC+ * DE-F+ /$
- C. $AB * C+D/E-F+$
- D. $ABCDEF*+/-+$

16. 若 $A=10$ 、 $B=4$ 、 $C=6$ 、 $D=4$ 、 $E=15$ 则后缀表达式“ $AB*CD+-E+$ ”的值为 ()

- A 45
- B 31
- C 53
- D 65

17. 已知后缀表达式 $4\ 5\ 2\ * + 2\ /\ 5\ 2\ - * 3\ @\ 8$ 的结果是 56, 则@处应该是什么操作符?

- A. +;
- B. -;
- C. *;
- D. ÷;

18. 利用栈求表达式的值时，设立运算数栈 OPEN。假设 OPEN 只有两个存储单元，则在下列表达式中，不会发生溢出的是()

- A. $A-B * (C-D)$
- B. $(A-B) * C-D$
- C. $(A-B * C)-D$
- D. $(A-B) * (C-D)$

19. 中缀表达式转换成后缀表达式，用栈来存放还不能确定运算次序的操作符。栈初始为空，下列中缀表达式中，转后缀表达式过程中用到栈空间最大的是 ()

- A. $A + B - C * (D + E) / F$
- B. $(A + B) * (C - D) / E + F$
- C. $A * B - (C + D) / (E - F)$
- D. $(A + B * (C - D)) / (E + F)$

20. 中缀表达式 $a+b-c$ 的前缀表达式为 ()

- A. $ab+c-$
- B. $+ -abc$
- C. $-+abc$
- D. $+a-bc$

21. $a * (b + c) - d$ 的前缀表达式是 ()

- A. $a b c d * + -$
- B. $a b c + * d -$
- C. $- * a + b c d$
- D. $- + * a b c d$

22. 以下关于栈和队列应用场景的描述，哪一个是正确的?()

- A. 在实现函数调用和递归过程中，通常使用队列来存储局部变量和返回地址
- B. 在实现图的广度优先搜索(BFS)算法时，通常使用栈作为辅助数据结构

- C.在实现括号匹配问题时，通常使用队列作为辅助数据结构
- D.在实现图的深度优先搜索(DFS)算法时，通常使用栈作为辅助数据结构

23. 用不带头结点的单链表存储队列时，其队头指针指向队头结点，其队尾指针指向队尾结点，则在进行删除操作时（）

- A.仅修改队头指针
- B.仅修改队尾指针
- C.队头和队尾指针都要修改
- D.队头和队尾指针都可能要修改

24. 某带链的队列初始状态为 $\text{front}=\text{rear}=\text{NULL}$ 。经过一系列正常的入队与退队操作后， $\text{front}=\text{rear}$ 。该队列中的元素个数为（）

- A.1
- B.0
- C.1 或 0
- D.不确定

25. 假设用 $Q[0..M]$ 实现环形队列， f 作为队头指针指向队头元素的前一个位置， r 作为队尾指针指向队尾元素。若用 $(r+1)\%(M+1) == f$ 作为队满的标志，则()

- A.可用 $f == r$ 作为队空的标志
- B.可用 $f > r$ 作为队空的标志
- C.可用 $(f+1) \% (M+1) == r$ 作为队空的标志
- D.队列中最多可以有 $M+1$ 个元素

26. 循环队列放在一维数组 $A[0..M-1]$ 中， front 指向队头元素， rear 指向队尾元素的后一个位置，队列中最多能容纳 $M-1$ 个元素，若当前队列有 k 个元素 ($0 < k < M-1$)，则下列关于入队操作(元素值为 value)和出队操作(保存到 q 中)，正确的是（）

- A. 入队操作: $A[++ \text{rear}] = \text{value}$; 出队操作: $q = A[\text{front} ++]$;
- B. 入队操作: $A[\text{rear} ++] = \text{value}$; 出队操作: $q = A[\text{front} ++]$;

C. 入队操作: $\text{rear} = (\text{rear} + 1) \% M$; $A[\text{rear}] = \text{value}$; 出队操作: $q = A[\text{front}]$; $\text{front} = (\text{front} + 1) \% M$;

D. 入队操作: $A[\text{rear}] = \text{value}$; $\text{rear} = (\text{rear} + 1) \% M$; 出队操作: $q = A[\text{front}]$; $\text{front} = (\text{front} + 1) \% M$;

27. 循环队列放在一维数组 $A[0..M-1]$ 中, end1 指向队头元素, end2 指向队尾元素。假设队列两端均可进行入队和出队操作, 队列中最多能容纳 $M-1$ 个元素。初始时空。下列判断队空和队满的条件中, 正确的是 ()。

A. 队空: $\text{end1} == \text{end2}$; 队满: $\text{end1} == \text{end2}$;

B. 队空: $\text{end1} == (\text{end2} + 1) \bmod M$; 队满: $\text{end1} == (\text{end2} + 1) \bmod (M - 1)$

C. 队空: $\text{end1} == (\text{end2} + 1) \bmod M$; 队满: $\text{end1} == (\text{end2} + 2) \bmod M$

D. 队空: $\text{end1} == \text{end2}$; 队满: $\text{end2} == (\text{end1} + 1) \bmod M$

28. 循环队列放在一维数组 $A[0, \dots, M-1]$ 中, front 指向队头元素, rear 指向队尾元素的最后一个位置。数组大小不可扩展, 并且不考虑额外开销的情况下, 该队列的剩余可用空间为 ()

A. $M - (\text{rear} - \text{front})$

B. $M - (\text{rear} - \text{front}) \% (M - 1)$

C. $M - (\text{rear} - \text{front} + M) \% M$

D. $M - 1 - (\text{rear} - \text{front} + M) \% M$

29. 若用一个大小为 6 的数组来实现环形队列, rear 作为队尾指针指向队列中尾部元素, front 作为队头指针指向队头元素的前一个位置。当前 rear 和 front 的值分别是 0 和 3, 当从队列中删除一个元素, 再加入两个元素后, rear 和 front 的值分别是()

A. 1 和 5

B. 2 和 4

C. 4 和 2

D. 5 和 1

30. 若循环队列使用数组 $A[0 \dots m-1]$ 存放数据元素, 已知头指针 $front$ 指向队首元素, 尾指针 $rear$ 指向队尾元素后的空单元, 则当前队列中的元素个数为()。

- A. $(rear-front+m) \% m$
- B. $rear - front + 1$
- C. $rear - front$
- D. $rear-front-1$

31. 已知循环队列存储在数组 $A[m]$ 中, $length$ 表示循环队列中的元素个数, $front$ 指向队首元素, $rear$ 指向队尾元素, 则 $front$ 所在位置应该是()。

- A. $rear - length$
- B. $(rear - length + m) \% m$
- C. $(rear+m+1-length) \% m$
- D. $(rear + length - 1) \% m$

32. 下列叙述中正确的是 ()

- A 在栈中, 栈顶指针的动态变化决定栈中元素的个数
- B 在循环队列中, 队尾指针的动态变化决定队列的长度
- C 在循环链表中, 头指针和链尾指针的动态变化决定链表的长度
- D 在线性链表中, 头指针和链尾指针的动态变化决定链表的长度

33. 一个队列只能从右侧入队, 左右侧皆可出队。顺序为 Ka 、 Kb 、 Kc 、 Kd 、 Ke 的序列入队后, 不能得到的输出是 ()

- A $Ka \rightarrow Kb \rightarrow Kc \rightarrow Kd \rightarrow Ke$
- B $Ke \rightarrow Kd \rightarrow Kc \rightarrow Kb \rightarrow Ka$
- C $Ka \rightarrow Kb \rightarrow Ke \rightarrow Kc \rightarrow Kd$
- D $Ke \rightarrow Ka \rightarrow Kc \rightarrow Kb \rightarrow Kd$

34. 若某队列允许在其两端进行入队操作, 但仅允许在一端进行出队操作, 允许入队出队操作交替进行。若入队序列为 a, b, c, d, e , 则下列出队序列中, 可能得到的有 ()

- I. b,a,c,d,e
- II. d,b,a,c,e
- III. d,b,c,a,e
- IV. b,c,a,d,e
- A. I、II、IV
- B. III、IV
- C. I、II
- D. II、IV

35. 设一个输入序列为 a,b,c,d, 借助一个输出受限的双向队列, 所得到的输出序列不可能是()

- A. d,a,c,b
- B. c,a,d,b
- C. a,b,c,d
- D. b,d,a,c

36. 在输入受限的双端队列中, 输入序列是 1, 2, ..., n, 输出序列是 P_1, P_2, P_3, P_4 , 若 $P_1 = n, P_2$ 的取值有 () 种

- A. n-1
- B. n-2
- C. 2
- D. 1

37. 有一个队列 Q 和一个栈 S, 已知入栈和入队的顺序都是 1,2,3,...,n($n>6$), 出栈或出队操作可以在任意一次入栈或入队或发生, 则下列说法错误的是 ()

- A、任何情况下出队的顺序都为 1,2,3,...,n
- B、如果第一个出栈的数字是 n-2, 则第二个出栈的数字可能是 n
- C、如果第一个出栈的数字是 n-3, 则第三个出栈的数字可能是 n-6
- D、如果第一个出栈的数字是 n-3, 则第三个出栈的数字可能是 n-4

38. 下列关于栈的说法错误的是 ()。

- A. 可以使用两个队列实现一个后入先出的栈
- B. 可以使用一个队列实现一个后入先出的栈
- C. 可以使用两个栈实现先入先出队列
- D. 可以使用一个栈实现先入先出队列

39. 设栈 S 和队列 Q 的初始状态为空,元素 e1-e6 依次通过栈 S,一个元素出栈后即入队列 Q,若 6 个元素出队的序列是 e2、e4、e3、e6、e5、e1,则栈 S 的容量至少应该是 ()

- A.5
- B.4
- C.3
- D.2

40. 数组 A[0...5][0...6]的每个元素占 5 个单元, 将其按列优先次序存储在起始地址为 1000 的连续内存单元中, 则元素 a[5] [5]的地址为 ()

- A. 1175
- B. 1180
- C. 1205
- D. 1210

41. 二维数组 M 的元素是 4 个字符(每个字符占一个存储单元)组成的串, 行下标 i 的范围从 0 到 4, 列下标 j 的范围从 0 到 5, M 按行存储时元素 M[3,5]的起始地址与 M 按列存储时元素 () 的起始地址相同,

- A. M[2, 4]
- B. M[3, 4]
- C. M[3, 5]
- D. M[4, 4]

42. 将一个 10×10 的对称矩阵 M 的下三角部分的元素 $M_{\{i,j\}}(1 \leq j \leq i \leq 10)$ 按列优先存入

C 语言的一维数组 N 中，元素类型为结构体， N 的首地址是 1000，按字节编址，结构体内部成员构成如下图所示，则元素 $M_{6,10}$ 的成员 b 所在的地址为 ()

```
C++
struct M{
    char a;
    int b;
    short c;
};
```

- A. 1604
- B. 1444
- C. 1532
- D. 1504

43. 设有一个 10 阶的对称矩阵 A ，采用压缩存储方式，按行优先存储其下三角部分，且设 $A[1][1]$ 为第一个元素，在 B 中存储于 $B[1]$ 中，那么 $A[8][5]$ 在 B 中的存放位置是 ()

- A. 13
- B. 32
- C. 33
- D. 40

44. 将一个 $n \times n$ 的对称矩阵 A 的下三角部分按行存放在一个一维数组 B 中， $A[0][0]$ 存放在 $B[0]$ 中，那么第 i 行的对角元素 $A[i][i]$ 在 B 中的存放位置是 ()

- A. $(i+3) * i / 2$
- B. $(i+1) * i / 2$
- C. $(2n-i+1) * i / 2$
- D. $(2n-i-1) * i / 2$

45. 下三角矩阵 $A(n \times n)$ 按列优先顺序压缩在数组 $S[(n+1) * n / 2]$ 中，若非零元素 a_{ij} 存放在 $S[k]$ 中，则 i, j, k 的关系为 ()。

- A. $k = i \times n + j$

- B. $k = (2n - j + 1) \times j / 2 + i - j$
- C. $k = (i + 1) \times i / 2 + j$
- D. $k = (2n - j + 1) \times j / 2 + i$

46. 设 A 是 $n \times n$ 的对称矩阵，将 A 的对角线及对角线上方的元素按以列为主的次序存放在一维数组 $A[1..n(n+1)/2]$ 中，对上述任一元素 $a_{ij}(1 \leq i, j \leq n, \text{ 且 } i \geq j)$ 在 B 中的位置为 ()

- A. $i(i-1)/2 + j$
- B. $j(j-1)/2 + i$
- C. $j(j-1)/2 + i - 1$
- D. $i(i-1)/2 + j - 1$

47. 稀疏矩阵的三元组存储方法()

- A. 实现转置运算很简单，只需将每个三元组的行标和列标交换
- B. 是一种链式存储方法
- C. 矩阵的非零元个数和位置在操作过程中变化不大时较有效
- D. 比十字链表法更高效

48. 已知一稀疏矩阵的三元组表为(1, 2, 3)、(1, 6, 1)、(3, 1, 5)、(3, 2, -1)、(5, 4, 5)、(5, 1, -3)，则其转置矩阵的三元组表中第 3 个三元组为 ()

- A. (2, 1, 3)
- B. (3, 1, 5)
- C. (3, 2, -1)
- D. (2, 3, -1)

49. 对稀疏矩阵采用压缩存储，其缺点之一是()

- A. 无法判断矩阵有多少行多少列
- B. 无法根据行列号查找某个矩阵元素
- C. 无法根据行列号计算矩阵元素的存储地址
- D. 使矩阵元素之间的逻辑关系更加复杂

50. 下列关于广义表(a, (b, c), ((d, e), f))的叙述错误的是 ()

- A. 表头是 a
- B. 表尾是 f
- C. 表的长度为 3
- D. 表的深度为 3

51. 串'ababaaababaa'的 next 数组值为()

- A. 01234567899
- B. 012121111212
- C. 011234223456
- D. 0123012322345

52. 串'ababaaababaa'的 next 数组为()。

- A. -1,0,1,2,3,4,5,6,7,8,8,8
- B. -1,0,1,0,1,0,0,0,0,1,0,1
- C. -1,0,0,1,2,3,1,1,2,3,4,5
- D. -1,0,1,2,-1,0,1,2,1,1,2,3

53. 串' ababaaababaa' 的 nextval 为 ()

- A. 0,1,0,1,1,2,0,1,0,1,0,2
- B. 0,1,0,1,1,4,1,1,0,1,0,2
- C. 0,1,0,1,0,4,2,1,0,1,0,4
- D. 0,1,1,1,0,2,1,1,0,1,0,4

54. 'aababaabac'的 nextval 数组为 ()

- A. -1,0,1,0,1,0,1,0,1,2,3,4
- B. -1,0,1,-1,1,-1,0,1,-1,4
- C. -1,-1,1,-1,1,-1,-1,1,-1,4
- D. -1,0,0,1,2,1,2,0,1,0

55. 下面关于串匹配算法的叙述中, 不正确的是 ()

- A.KMP 算法的特点是在模式匹配过程中指示主串的指针不会变小
- B.设主串长度为 n , 模式串长度为 m , 当 $n \leq m$ 时, 简单的匹配算法所花的时间代价可能会比 KMP 算法少
- C.设主串长度为 n , 模式串长度为 m , 则串匹配的 KMP 算法的时间复杂度为 $O(m+n)$
- D.KMP 算法中, 求模式串的 next 数组值时, 和相匹配的主串有关

56. 已知字符串 S 为"aabaacaadaafa", 模式串 P 为"aada"。采用 KMP 算法结合 (nextval)数组进行匹配, 匹配成功时共进行 () 次比较。

- A. 11
- B. 12
- C. 13
- D. 14

57. 下列关于 kmp 算法的说法, 错误的是 ()

- I. kmp 算法的时间复杂度更优, 且在主串与子串有很多“部分匹配”时效率相对于普通匹配算法才提升显著
- II. kmp 主要优点是主串不需要回溯, 因此 kmp 算法在每一次匹配过程中, 主串指针不可能变大。
- III. 'aaaa'的 next 数组为'-1,0,2,3'
- IV. 失配时只需简单把子串指针 j 的值改变为 $next[j]$ 即可

- A. I、III
- B. II、IV
- C. II、III、IV
- D. III、IV

58. 对于主串 $S = \text{"aabaabaabaac"}$, 模式串 $T = \text{"aabaac"}$, 在使用 kmp 算法进行字一共发生了 () 次“失配”的情况($S[i] \neq T[j]$ 即发生了失配), 最后字符串匹配时, 一次“失配”时,

模式串失配处的字符是()

- A.3, b
- B.2, c
- C.3, c
- D.2, b

59. 设主串 $T = \text{'abaabaabacacaabaabcc'}$, 模式串 $S = \text{'abaabc'}$, 采用 KMP 算法进行模式匹配, 到匹配成功为止, 在匹配过程中模式串右滑的总距离是 () (模式串首个字符失配计右滑 1 格)

- A.10
- B.11
- C.12
- D.13

五 树

1. 在一棵度为 4 的树 T 中，若有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的叶结点个数是 ()。
A. 41 B. 82 C. 113 D. 122
2. 在一棵度为 4 的树中，结点总数为 20 个，有 2 个度为 3 的结点，1 个度为 2 的结点，7 个度为 1 的结点，那么叶子结点有 () 个
A. 1
B. 2
C. 8
D. 9
3. 对于一棵具有 n 个节点、度为 4 的树来说()
A. 树的高度至多是 $n-3$
B. 树的高度至多是 $n-4$
C. 第 i 层上至多有 4^{i-1} 个节点
D. 至少在某一层上正好有 4 个节点
4. 具有 n 个结点的 m 叉树用 m 叉链表表示，则树中空指针域的个数为 ()
A. $mn + 1$
B. $(m - 1)n + 1$
C. $(n - 1)m + 1$
D. $mn - 1$
5. 4 个结点二叉树的树型有 () 种
A. 13

- B. 14
- C. 15
- D. 16

6. 8 个结点且高度为 8 的二叉树的树型有 () 种

- A. 4
- B. 8
- C. 256
- D. 128

7. 下面关于二叉树的结论正确的是 ()

- A. 二叉树中, 度为 0 的结点个数等于度为 2 的结点个数加 1
- B. 二叉树中结点个数必大于 0
- C. 完全二叉树中, 任何一个结点的度, 或者为 0, 或者为 2
- D. 二叉树的度是 2

8. 每个结点的度都是 0 或 2 的二叉树称正则二叉树, n 个结点的正则二叉树中有 () 个叶结点。

- A. $\lfloor \log_2 n \rfloor$
- B. $(n - 1)/2$
- C. $\lfloor \log_2(n + 1) \rfloor$
- D. $(n + 1)/2$

9. 设有 $n(n \geq 1)$ 个结点的二叉树采用三叉链表表示, 其中每个结点包含三个指针, 分别指向其左孩子、右孩子及双亲(若不存在, 则置为空), 则下列说法中正确的是()。

- I. 树中空指针的数量为 $n+2$
- II. 所有度为 2 的结点均被三个指针指向
- III. 每个叶结点均被一个指针所指向

- A. I

- B. I、II
- C. I、III
- D. II、III

10. 设高度为 h 的二叉树只有度为 0 和度为 2 的结点, 则此类二叉树中所包含的结点数至少为()

- A. $h+1$
- B. $2h-1$
- C. $2h$
- D. $2h+1$

11. 下列说法错误的是 ()

- A. 不存在这样的二叉树: 它有 n 个度为 0 的结点, $n-1$ 个度为 1 的结点, $n-2$ 个度为 2 的结点
- B. 在任何一棵完全二叉树中, 终端结点或者和分支结点一样多, 或者只比分支结点多一个
- C. 若一个叶子节点是某二叉树先序遍历序列中的最后一个结点, 则它必是该树后序遍历序列中的最后一个结点
- D. 只要知道完全二叉树中结点的先序序列, 就可以唯一地确定它的逻辑结构

12. 在一个用数组表示的完全二叉树中, 根结点的下标为 1, 那么下标为 17 和 19 的结点的最近公共祖先的下标是()。

- A. 1
- B. 2
- C. 4
- D. 8

13. 一个具有 513 个结点的二叉树, 有()种可能的层高

- A 513
- B 512
- C 504

D 503

14. 给定结点个数 n ，在下面二叉树中，叶结点个数不能确定的是()

- A. 满二叉树
- B. 完全二叉树
- C. 哈夫曼树
- D. 二叉排序树

15. 在一棵二叉树中有 30 个叶子结点，仅有一个孩子的结点有 20 个，则该二叉树共有结点()

- A. 79
- B. 76
- C. 56
- D. 81

16. 已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，则完全二叉树的结点个数最多为 ()

- A. 39
- B. 52
- C. 111
- D. 119

17. 一棵有 15 个节点的完全二叉树和一棵同样有 15 个节点的普通二叉树，叶子节点的个数最多会差()个

- A 3
- B 5
- C 7
- D 9

18. 设有 n ($n > 1$) 个结点的二叉树采用三叉链表表示, 其中每个结点包含三个指针, 分别指向左孩子、右孩子及双亲 (若不存在, 则置为空), 则下列说法正确的有 ()

I. 树中空指针数量为 $n+2$

II. 通过三叉链表, 可以在 $O(1)$ 时间复杂度找到二叉树中任意结点前中后序遍历的直接前驱。

III. 每个叶子结点只有一个指针指向它

IV. 任意度为 2 的分支结点的三个指针非空

A. II、III、IV

B. I、III、IV

C. I、III

D. I、II、IV

19. 如果一棵二叉树的先序序列是...a...b..., 中序序列是...b...a..., 则 ()

A. 节点 a 和节点 b 分别在某节点的左子树和右子树中

B. 节点 b 在节点 a 的右子树中

C. 节点 b 在节点 a 的左子树中

D. 节点 a 和节点 b 分别在某节点的两棵非空子树中

20. 在下列关于二叉树遍历的说法中, 正确的是()。

A. 若有一个结点是二叉树中某个子树的中序遍历结果序列的最后一个结点, 则它一定是该子树的前序遍历结果序列的最后一个结点

B. 若有一个结点是二叉树中某个子树的前序遍历结果序列的最后一个结点, 则它一定是该子树的中序遍历结果序列的最后一个结点

C. 若有一个叶结点是二叉树中某个子树的中序遍历结果序列的最后一个结点, 则它一定是该子树的前序遍历结果序列的最后一个结点

D. 若有一个叶结点是二叉树中某个子树的前序遍历结果序列的最后一个结点, 则它一定是该子树的中序遍历结果序列的最后一个结点

21. 设 n, m 为一棵二叉树上的两个结点, 在后序遍历时, n 在 m 前的充分条件是()

A. n 在 m 右方

B. n 是 m 祖先

C.n 在 m 左方

D.n 是 m 子孙

22. 先序序列为 1,2,3,4 的不同二叉数有 () 个

A. 13

B. 14

C. 15

D. 16

23. 4 个不同权值大小不同的元素组成的二叉排序树, 有 () 种不同的树型

A. 13

B. 14

C. 15

D. 16

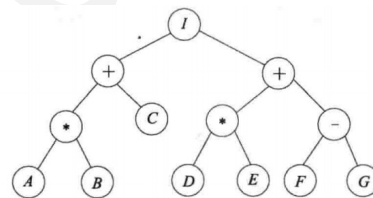
24. 设有一表示算术表达式的二叉树(见下图), 它所表示的算术表达式是()。

A. $A * B + C / (D * E) + (F - G)$

B. $(A * B + C) / (D * E) + (F - G)$

C. $(A * B + C) / (D * E + (F - G))$

D. $A * B + C / D * E + F - G$



25. 在二叉树中有两个结点 m 和 n, 若 m 是 n 的祖先, 则使用()可以找到从 m 到 n 的路径。

A. 先序遍历

B. 中序遍历

C. 后序遍历

D. 层次遍历

26. 下列关于树的说法错误的是 () 。

I. 树的层序遍历一般使用栈

II. 图的深度优先遍历必须使用栈来实现

III. 图的广度优先遍历必须使用栈

IV. 树的后序遍历非递归实现中, 栈内的结点元素是当前遍历到的结点的祖先元素集合

A. I、III、IV

B. II、IV

C. I、II

D. I、III

27. 一棵二叉树的前序遍历序列为 ABCDEFG, 它的中序遍历序列可能是 ()

A. CABDEFG

B. ABCDEFG

C. DACEFBG

D. DBCEAFG

28. 由 6 个节点组成的二叉树, 若中序遍历序列为 ABCDEF, 则不可能的后序遍历序列是 ()

A. CBEADF

B. ADFECB

C. ABDECF

D. BADCFE

29. 某二叉树的前序序列为 ABCDEFG, 中序序列为 DCBAEFG 则该二叉树的深度 (根结点在第 1 层) 为 ()

A. 2

B. 3

C. 4

D. 5

30. 对于层序序列 (ABDCEFGIH) 和中序序列 (BCAEDGHFI) 所确定的二叉树, 下列选项正确的是 ()

A. 后序序列是 CBEGHIFDA

B. 先序序列是 ABCDEFGIH

C. 后序序列是 CBEIHGFDA

D. 先序序列是 ABCDEFGHI

31. 一棵二叉排序树的后序遍历序列是 15,10,23,25,20,35,42,39,30。请给出这棵树的前序遍历序列()

- A.30.20.15.10.23.25.39.35,42
- B.30,20,10,15,23,25,39,35,42
- C.30.20.10.15.25.23.39.35.42
- D.30.20.10.15.23.25.35.39.42

32. 下列关于二叉树的遍历的陈述中, 正确的是 ()。

- I. 存在一棵二叉树的叶结点在先序、中序、后序遍历序列中相对位置可能不同
- II. 要交换二叉树的所有分支结点的左右子树的位置, 利用中序遍历框架解决最合适
- III. 含有 4 个结点的二叉树最多有 14 种形态
- IV. 一棵非空的二叉树的前序序列和中序序列正好相反, 则该二叉树一定只有一个叶结点

- A. III、IV B. II、III、IV C. II、III D. IV

33. 在二叉树的前序序列、中序序列和后序序列中, 所有叶结点的先后顺序()。

- A. 都不相同
- B. 完全相同
- C. 前序和中序相同, 而与后序不同
- D. 中序和后序相同, 而与前序不同

34. 某二叉树的先序遍历序列和后序遍历序列正好相反, 则关于该二叉树一定是 ()

- A. 空或只有一个结点
- B. 该二叉树转成森林, 每棵树只有一个根节点
- C. 度为 1 的树
- D. 高度等于其结点数

35. 某二叉树的中序遍历序列和后序遍历序列正好相同, 则关于该二叉树一定是 ()

- A. 空树或任一结点没有左子树
- B. 空树或任一结点没有右子树
- C. 空树或每个结点的度为 1
- D. 空树或满二叉树

36. 一棵完全二叉树的中序遍历序列为 1,2,3,4,5,6,7, 那么其先序遍历为 ()

- A. 4,2,1,3,6,5,7
- B. 4,2,6,1,3,5,7
- C. 1,3,2,5,7,6,4
- D. 7,6,5,4,3,2,1

37. 若一棵二叉树的先序遍历序列为 a,e,b,d,c,后序遍历序列为 b,c,d,e,a, 则下列说法错误的是 ()

- A. 根节点只有一个孩子
- B. 度为 2 的结点只有一个
- C. b 一定是 e 的左孩子
- D. c 一定是 d 的左孩子

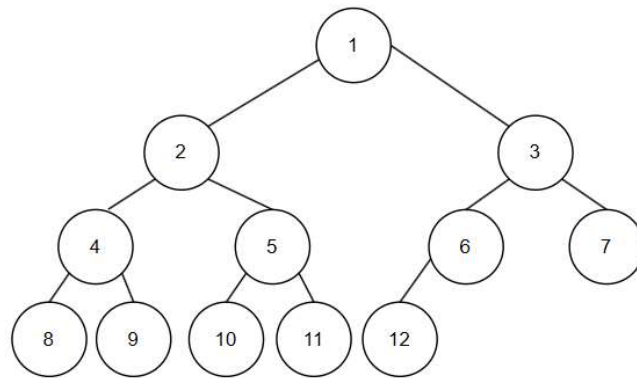
38. 任何一棵二叉树的叶子结点在先序、中序和后序遍历序列中的相对次序 ()

- A. 不发生改变
- B 发生改变
- C.不能确定
- D.以上都不对

39. 设结点 x 和结点 y 是二叉树 T 中的任意两个结点, 若在先序序列中 x 在 y 之前, 而在后序序列中 x 在 y 之后, 则 x 和 y 的关系是()

- A. x 是 y 的左兄弟
- B.x 是 y 的右兄弟
- C.x 是 y 的祖先
- D.x 是 y 的后代

40. 队列常用于二叉树的层序遍历，下列二叉树的层序遍历中，队列空间的大小至少是 ()



- A. 4
- B. 5
- C. 6
- D. 7

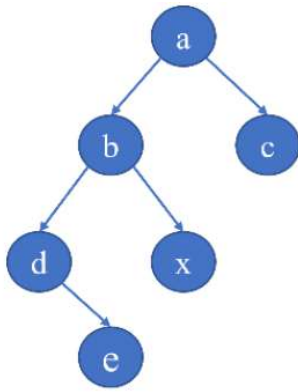
41. 在下列叙述中，正确的叙述是 ()

- A. 树的先序遍历和中序遍历可以得到树的后遍历
- B. 将一棵树转换成二叉树后，根结点没有右子树
- C. 采用二叉链表作存储结构，树的先序遍历和其相应的二叉树的先序遍历的结果不同
- D. 一棵树中的叶子结点数一定等于与其对应的二叉树的叶子结点数

42. 一棵具有 n 个非叶子结点完全二叉树的线索树，含有()条线索

- A. $2n+1$ 或 $2n$
- B. $2n+2$ 或 $2n+1$
- C. $2n+1$ 或 $2n-1$
- D. $2n+2$ 或 $2n-2$

43. 判断线索二叉树中 *p 结点有右孩子结点的条件是()。
- A. $p \neq \text{NULL}$
 - B. $p \rightarrow \text{rchild} \neq \text{NULL}$
 - C. $p \rightarrow \text{rtag} == 0$
 - D. $p \rightarrow \text{rtag} == 1$
44. 知一棵二叉树的后序序列为 DABEC，中序序列为 DEBAC，则其对应的二叉树的先序线索二叉树中，右指针为线索()的结点有 () 个。
- A. 1
 - B. 2
 - C. 3
 - D. 4
45. 下列关于线索二叉树，说法不正确的是 ()
- A. 在中序线索二叉树中，若某结点有右孩子，那么其后继结点一定它右子树的最左下结点
 - B. 在先序线索二叉树中，若某结点有左孩子，那么可能无法通过该结点的指针找到他的前驱结点
 - C. 线索二叉树中，若总结点数为 n ，那么表示线索的指针有 $n+1$ 个
 - D. 在后序线索二叉树中，若一个非根结点没有右孩子，那么它的右指针一定指向其父亲结点
46. 二叉树在线索化后，仍不能有效求解的问题是()。
- A. 先序线索二叉树中求先序后继
 - B. 中序线索二叉树中求中序后继
 - C. 中序线索二叉树中求中序前驱
 - D. 后序线索二叉树中求后序后继
47. 若对以下二叉树进行后序线索化，则结点 x 的左、右线索分别指向()



- A . e, a
- B . d, b
- C . e, b
- D . b, c

48. () 的遍历仍需要栈的支持。

- A、前序线索树
- C、后序线索树
- B、中序线索树
- D、所有线索树

49. 下列说法错误的是 () 。

- A. 若一棵二叉排序树中一个结点有两个孩子，则它的中序后继结点没有左孩子，它的中序前驱结点没有右孩子
- B. 若二叉排序树中一个结点 x 的右子树为空，且 x 有一个后继 y ，则 y 一定是 x 的祖先，且其左孩子也是 x 的祖先（结点本身也视为自己的一个祖先）
- C. 中序线索树中，从最左边的结点开始不断地查找右线索，不一定能遍历树内所有的结点
- D. 若 x 是二叉排序树的叶结点， y 是其父结点，那么 y 的数值要么是树中大于 x 的数值的最小关键字，要么是树中小于 x 的数值的最大关键字

50. 下列说法正确的是 ()

- A. 在二叉树中，具有两个孩子的双亲节点，在中序遍历序列中，它的后继节点(后继节

点是指中序遍历序列中排在某节点之后的节点)中最多只能有一个孩子节点。

B. 在二叉树中，具有一个孩子的双亲节点，在中序遍历序列中，它没有后继孩子节点

C. 已知二叉树的先序遍历和后序遍历并不能唯一地确定这棵树，因为不知道树的根节点是哪一个

D. 将一棵树转换成二叉树后，根节点没有左子树

51. 设某树的孩子兄弟链表示中共有 6 个空的左指针域、7 个空的右指针域，包括 5 个结点的左、右指针域都为空，则该树中叶结点的个数是()。

A. 7

B. 6

C. 5

D. 不能确定

52. 下列关于森林与二叉树的转换，说法错误的是 ()

A. 二叉树中没有右孩子的结点个数等于原森林中分支结点的个数+1

B. 当且仅当森林只有一棵树时，二叉树的分支数和原森林的分支数相同

C. 森林中的叶子结点数量等于二叉树中空左指针数量。

D. 二叉树中，某个结点的左右孩子在原森林中一定是叔侄关系。

53. 已知二叉树 T 的顺序存储为{1, 10, 6, 3, 2, 22, 4, -1, -1, -1, -1, -1, 5, -1, 8}(-1 代表空结点)，那么二叉树 T 对应的森林 F 的先根遍历序列为()

A. 3, 10, 1, 2, 22, 6, 5, 4, 8

B. 1, 10, 3, 2, 6, 22, 5, 4, 8

C. 1, 10, 2, 3, 6, 22, 5, 4, 8

D. 1, 2, 10, 3, 6, 22, 5, 4, 8

54. 一棵二叉排序树的层序遍历序列是 30,20,39,10,23,35,42,15,25。这棵二叉树转换成的树/森林的后根遍历为()

A. 10,15,25,23,20,35,42,39,30

B. 30,20,10,15,23,25,39,35,42

- C. 15,10,23,25,20,35,42,39,30
- D. 10,15,20,23,25,30,35,39,42

55. 若 T' 是由有序树 T 转换而来的二叉树, 则 T 中结点的后根序列就是 T' 中结点的() 序列。

- A. 先序
- B. 中序
- C. 后序
- D. 层序

56. 如果用孩子兄弟链来表示一棵具有 $n(n>1)$ 个节点的树, 则在该存储结构中()

- A. 至多有 $n-1$ 个非空的右指针域
- B. 至少有两个空的右指针域
- C. 至少有两个非空的左指针域
- D. 至多有 $n-1$ 个空的右指针域

57. 如果森林 F 采用“孩子-兄弟”表示法对应的二叉树是 16 个结点的完全二叉树, 问: 森林 F 中树的数目和最大树的结点个数分别为()。

- A. 2 和 8
- B. 2 和 9
- C. 4 和 8
- D. 4 和 9

58. 设森林 F 中有 3 棵树, 第一、第二、第三棵树的结点个数分别为 $M_{\{1\}}$, $M_{\{2\}}$ 和 $M_{\{3\}}$ 。将森林 F 转换成对应的二叉树 T , 若以第一棵树的根结点作为 T 的根结点, 则 T 的根结点的右子树上的结点个数是 ()

- A. M_1
- B. $M_1 + M_3$
- C. M_3
- D. $M_2 + M_3$

59. 采用双亲表示法描述一棵树，则具有 n 个结点的树至少需要 () 个指向双亲的指针。
- A. n
 - B. $n - 2$
 - C. $n - 1$
 - D. $2n$
60. 若某个森林转换的二叉树是一棵高度为 h 的满二叉树，那么关于原森林说法错误的是 ()
- A. 有 h 棵树
 - B. 分支结点有 $2^{h-1} - 1$
 - C. 第 i 棵树的结点个数是 2^{h-i}
 - D. 第 i 棵树的度为 $h-i-1$
61. 若一棵树有 4 个结点，那么其所有可能的树型有 () 种
- A. 4
 - B. 5
 - C. 14
 - D. 24
62. 已知森林 F 对应的二叉树为 T ，且该二叉树是一棵 BST，其后序序列是 1,3,4,2,7,9,8,10,6,5，那么在森林 F 中说法错误的是 ()
- A. 有 3 棵树
 - B. 第一棵树的后根遍历是升序序列
 - C. 森林的度为 2
 - D. 原森林的先根遍历是 5,2,1,3,4,6,10,8,7,9
63. 某二叉树结点的中序序列为 BDAECF,后序序列为 DBEFCA，则该二叉树对应的森林包括()棵树。
- A. 1
 - B. 2
 - C. 3

D. 4

64. 设 F 是由 T_1 、 T_2 和 T_3 三棵树组成的森林，与 F 对应的二叉树为 B ， T_1 、 T_2 和 T_3 的结点数分别为 N_1 、 N_2 和 N_3 ，则二叉树 B 的根结点的左子树的结点数为()

A N_1-1

B N_2-1

C N_2+N_3

D N_1+N_3

65. 如果在树的孩子兄弟链存储结构中有 6 个空的左指针域，7 个空的右指针域，5 个节点左右指针域都为空，则该树中叶子节点有 () 个

A.7

B.6

C 5

D.4

66. 下列说法错误的是 ()

A 哈夫曼树是带权路径长度最短的树，路径上权值较大的节点离根较近。

B 在先序遍历二叉树的序列中，任何节点其子树的所有节点都是直接跟在该节点之后的。

C 存在这样的二叉树，对它采用任何次序的遍历，结果都相同。

D 若一个叶子节点是某二叉树先序遍历序列中的最后一个节点，则它必是该树中序遍历序列中的最后一个节点。

67. 下列说法正确的是 ()

I. 所有叶节点深度一致的二叉树，必为完全二叉树。

II. 完全二叉树的子树，也一定是完全二叉树。

III. 在 Huffman 算法过程中，权重小的内部节点必然早于权重大的内部节点被创建。

IV. 由同一组互异关键码，按不同次序逐个插入而生成的 BST 必互异。

V. 在 AVL 树中删除节点之后若树高降低，则必然做过旋转调整

A. I、II、III、IV

B. II、III、IV、V

C. II、III

D. II、III、V

68. 有 5 个字符，根据其使用频率设计对应的哈夫曼编码，以下哪个是不可能的哈夫曼编码 ()

A. 000, 001, 010, 011, 1

B. 0000, 0001, 001, 01, 1

C. 000, 001, 01, 10, 11

D. 00, 100, 101, 110, 111

69. 已知字符集{a, b, c, d, e, f}，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是 ()。

A. 00, 1011, 01, 1010, 11, 100

B. 00, 100, 110, 000, 0010, 01

C. 10, 1011, 11, 0011, 00, 010

D. 0011, 10, 11, 0010, 01, 000

70. 下列关于哈夫曼树和哈夫曼编码的陈述中，正确的是 ()。

A. 一个数据文件由 8 位定长度的字符组成，字符集内所有的 256 个字符出现的频率大致相同，最高的频率低于最低频率的 2 倍，那么此时哈夫曼编码比 8 位固定长度编码更有效

B. 哈夫曼树中度为 1 的结点数等于度为 2 和度为 0 的结点数之差

C. 哈夫曼树的结点总数不能是偶数

D. 若从二叉树的任一结点出发，到根结点的路径上所经结点的序列按其关键字有序，则该二叉树一定不是一棵哈夫曼树

71. 以下说法错误的是()

- A 一般在哈夫曼树中，权值越大的叶子离根结点越近
- B 哈夫曼树中没有度数为 1 的分支结点
- C 若初始森林中共有 n 棵二叉树，最终求得的哈夫曼树共有 $2n-1$ 个结点
- D 若初始森林中共有 n 棵二叉树，进行 n 次合并后才能剩下一棵最终的哈夫曼树

72. 假设某段通信电文仅由 6 个字母 ABCDEF 组成，字母在电文中出现的频率分别为 2, 3, 7, 15, 4, 6。根据这些频率作为权值构造哈夫曼编码，最终构造出的哈夫曼树带权路径长度与字母 B 的哈夫曼编码分别为(这里假定左节点的值小于右节点的值)()

- A.86, 1011
- B.70, 1000
- C.86, 0001
- D.70, 0010

73. 下列选项给出的是从根分别到达两个叶结点路径上的权值序列，能属于同一棵哈夫曼树的是()。

- A.26, 16, 5 和 24, 10, 6
- B.26, 16, 8 和 26, 10, 5
- C.26, 10, 5 和 24, 12, 7
- D.26, 16, 7 和 24, 16, 8

74. 下列说法正确的是 ()

- A. 哈夫曼树是带权路径长度最小的二叉树。
- B. 哈夫曼树是完全二叉树
- C. 二叉树是度为 2 的有序树。
- D. 在任意一棵二叉排序树中删除一个分支节点，接着又将该结点插入到二叉排序树

中，则所得到的二叉排序树和删除前的二叉排序树相同。

75. 若度为 m 的哈夫曼树中，叶结点个数为 n ，则非叶结点的个数为 ()

- A. $n-1$
- B. $\lceil n/m \rceil - 1$
- C. $\lceil (n-1)/(m-1) \rceil$
- D. $\lceil n/(m-1) \rceil - 1$

76. 并查集中最核心的两个操作是:①查找，查找两个元素是否属于同一个集合;②合并，若两个元素不属于同一个集合，且所在的两个集合互不相交，则合并这两个集合。假设初始长度为 10(0~9)的并查集，按 1-2、3-4、5-6、7-8、8-9、1-8、0-5、1-9 的顺序进行合并操作，最终并查集共有()个集合。

- A.1
- B.2
- C.3
- D.4

77. 下列关于并查集的说法中，正确的是()。

- A.并查集不能检测图中是否存在环路的问题
- B.通过路径优化后的并查集在最坏情况下的高度仍是 $O(n)$
- C.Find 操作返回集合中元素个数的相反数，它用来作为某个集合的标志
- D.并查集基于树的双亲表示法

78. 下列关于并查集的叙述中，()是错误的。

- A.并查集是用双亲表示法存储的树
- B.并查集可用于实现克鲁斯卡尔算法
- C.查集可用于判断无向图的连通性
- D.在长度为 n 的并查集中进行查找操作的时间复杂度为 $O(\log_2 n)$

79. 下列关于并查集的说法，错误的是（）

I. 并查集采用的是树的双亲表示法

II. 一个集合中根结点的双亲域的绝对值表示集合中的结点个数

III. find()操作返回的一定是负值

IV. 改进的 Union()操作使其本身的时间复杂度降低了

V. 在 Union 改进后，进一步，Find()经过压缩路径优化后，可使集合树的深度为 $O(\log_2 n)$

A. III、V

B. IV、V

C. II、III

D. III、IV、V

六 图

1. 连通的无向图 G 结点情况：16 个度为 4 的顶点，2 个度为 3 的顶点，5 个度为 2 的顶点，4 个度为 1 的结点。则 G 的边数有 () 条。

- A. 84
- D. 42
- C. 27
- D. 64

2. 设有无向图 $G=(V, E)$ 和 $G'=(V', E')$ ，若 G' 是 G 的生成树，则下面说法不正确的是 ()

- A. G' 为 G 的连通分量
- B. G' 是 G 的无环子图
- C. G' 为 G 的子图
- D. G' 为 G 的极小连通子图且 $V'=V$

3. 下列关于有向图强连通分量特性的叙述中，正确的是 ()。

- A. 若有向图 G 有两个不同的强连通分量 c 和 c' ，设顶点 $u, v \in c$ ，顶点 $u', v' \in c'$ ，若存在一条从 u 到 u' 的路径 $u \rightarrow u'$ ，则可能存在一条从 v' 到 v 的路径 $v' \rightarrow v$
- B. 若在图 G 中加入一条新边，若该边的两个顶点都处于同一个连通分量，则可能会增加 G 的连通分量的个数
- C. 若在图 G 中加入一条新边，若该边的两个顶点 A 和 A' 分别处于不同的连通分量，并且 A 和 A' 之间原先没有路径，则添加后可能会增加 G 的连通分量的个数
- D. 若在图 G 中加入一条新边，若该边的两个顶点 A 和 A' 分别处于不同的连通分量，并且 A 和 A' 之间原先有一条路径，则添加后可能会减少 G 的连通分量的个数

4. 一个有 n 个顶点和 n 条边的无向图一定是()。

- A. 连通的
- B. 不连通的
- C. 无环的

D.有环的

5. 下列关于有向图强连通分量特性的叙述中, $\langle u, v \rangle$ 表示顶点 u 到顶点 v 的有向弧, 下列说法正确的是 ()。

A. 若有向图 G 有两个不同的强连通分量 c 和 c' , 设顶点 $u \in c$, 顶点 $u' \in c'$, 那么不可能存在弧 $\langle u, u' \rangle$

B. 若有向图 G 有强连通分量 c , $u, v \in c$, 若删除弧 $\langle u, v \rangle$, 有向图 G 的强连通分量一定增加。

C. 若有向图 G 有两个不同的强连通分量 c 和 c' , 设顶点 $u, v \in c$, 顶点 $u', v' \in c'$, 若存在 $u \rightarrow u'$ 的路径, 那么一定不存在 $\langle v', v \rangle$

D. 若有向图 G 有两个不同的强连通分量 c 和 c' , 设顶点 $u \in c$, 顶点 $u' \in c'$, 若不存在 $\langle u, u' \rangle$, 那么添加 $\langle u', u \rangle$ 后, 该图的强连通分量一定不变

6. 一个有 28 条边的非连通无向图至少有()个顶点。

A. 7

B. 8

C. 9

D. 10

7. 有 31 条弧的强连通有向图至少有 () 个顶点

A. 6

B. 7

C. 8

D. 9

8. 有向图的结点数为 4, 则至少需要多少 () 条边满足在任意条件下这个图是强连通图。

A. 7

B. 8

C. 9

D. 10

9. 如果 G 的强连通分量的数量等于 $k > 1$, 结点总数是 n , 那么边数 m 应该满足 ()

A. $m \geq n - 1$

B. $m \geq n - k - 1$

C. $m \leq n^2$

D. $m \leq (n-1)^2$

10. 具有 51 个顶点和 21 条边的无向图的连通分量最多为()。

A. 33

B. 34

C. 45

D. 32

11. 下列关于图的陈述说法错误的是 () 。

A. 强连通的有向图的任何顶点到其他所有顶点都有弧

B. 有向完全图一定是强连通有向图

C. DFS 生成树的高度一般不小于 BFS 生成树的高度

D. 当各边上的权值相同时, BFS 算法可用来解决单源最短路径问题

12. 若具有 n 个顶点的图是一个环, 则它有()棵不同的生成树。

A. n^2

B. n

C. $n-1$

D. 1

13. 若一个具有 n 个顶点、 e 条边的无向图是一个森林, 则该森林中必有()棵树。

A. n

B. e

C. $n-e$

D. 1

14. 考虑一个有向图 G , 假设该图中没有自环。以下关于有向图 G 的描述, 错误的是 () 。

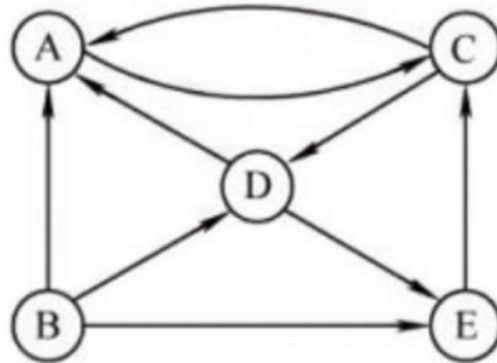
A. 如果 G 的强连通分量的数量等于 $k < n$, 结点总数是 n , 那么边数 $m \geq n$

B. 如果 G 是一个有向树, 那么 $m = n-1$

C. 如果 G 是一个强连通图, 那么 $m \geq n$

D.如果 G 是一个有向完全图, 那么 $m=n(n-1)$

15. 在如下图所示的有向图中, 共有()个强连通分量。



- A.1
- B.2
- C.3
- D.4

16. 下列关于图的叙述, 正确的是()

- I, 任意无向图的生成森林中顶点数量=边数+1
 - II, 在一个连通无向图中, 顶点数为 n , 边数为 m , 则它的生成树的边数为 $n-1$
 - III, 对于有向无环图, 可以通过拓扑排序来判断两个顶点之间是否有边
 - IV, 在一个无向图中, 若从任意一个顶点出发都可以到达其他所有顶点, 则这个图是连通图
- A.仅 II, IV
 - B.仅 I、III
 - C.仅 I、II、IV
 - D.仅 II、III、IV

17. 设 G 是一个非连通无向图, 有 10 条边, 则该图的顶点数至少有()个。

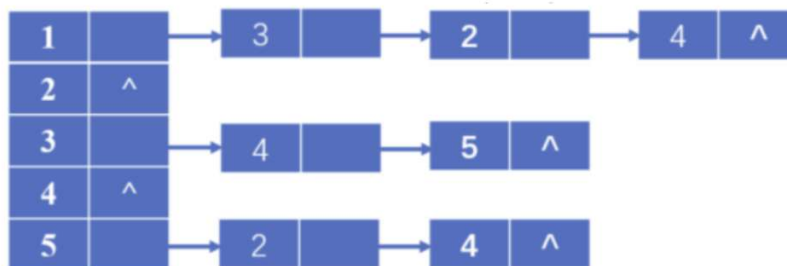
- A. 5
- B. 6
- C. 7

D. 8

18. 下列关于图的说法正确的是()

- A. 无向连通图的极小连通子图就是它本身
- B. 对有向图做 dfs 时, dfs 的调用次数与其连通分量的数量无关
- C. 一棵非空二叉树可以看作一个无向连通图
- D. 邻接矩阵为上三角矩阵的图一定存在唯一的拓扑排序

19. 已知一个有向图的邻接表存储结构如右图所示, 根据有向图的深度优先遍历算法, 从顶点 1 出发, 所得到的顶点序列是()



- A. 1, 2, 3, 5, 4
- B. 1, 2, 3, 4, 5
- C. 1, 3, 4, 5, 2
- D. 1, 4, 3, 5, 2

20. 关于有向图的描述, 正确的是()

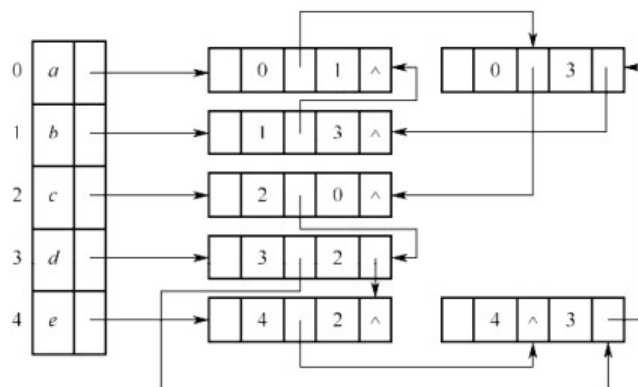
- A. 从邻接矩阵中删除一个顶点的时间复杂度是 $O(1)$
- B. 有向图的入度和出度之和等于边数的两倍
- C. 有向图中的强连通分量可能包含多个非连通子图
- D. 在有向无环图中, 可以通过拓扑排序得到唯一的线性序列

21. 设图的邻接矩阵 A 如下所示, 各顶点的入度依次是()。

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

- A. 1,2,1,2
- B. 2,2,1,1
- C. 3,4,2,3
- D. 1,2,3,4

22. 若无向图 $G=(V,E)$ 的邻接多重表如下图所示，则不属于该图的广度优先遍历序列的是()。



- A. abdce
- B. decab
- C. bacde
- D. ecdab

23. 一个含有 n 个顶点的连通且无环的简单无向图，在其邻接矩阵存储结构中中共有()个零元素

- A $n^2 - 2n$
- B $n^2 - 2n - 2$
- C $n^2 - 2n + 2$
- D $2(n - 1)$

24. 带权有向图 G 用邻接矩阵 A 存储，则顶点 i 的入度等于 A ()

- A.第 i 行非 ∞ 的元素之和
- B.第 i 列非 ∞ 的元素之和
- C.第 i 行非 ∞ 且非 0 的元素个数
- D.第 i 列非 ∞ 且非 0 的元素个数

25. 下列关于邻接表和邻接矩阵的陈述中, 错误的是 ()。

- A. 求有向图的结点的入度必须遍历整个邻接表
- B. 在有向图的邻接表中, 某个顶点 v 在链表中出现的次数等于该顶点的入度
- C. 若邻接矩阵中的主对角线以下的元素为 0, 则该图必定存在一个拓扑排序序列
- D. 因为有向无环图具有拓扑序列, 所以它的邻接矩阵必定是上三角矩阵或下三角矩阵

26. 下面关于图的存储的叙述中正确的是()

- A. 用邻接表法存储图, 占用的存储空间大小只与图中边数有关, 而与结点个数无关
- B. 用邻接表法存储图, 占用的存储空间大小与图中边数和结点个数都有关
- C. 用邻接矩阵法存储图, 占用的存储空间大小与图中结点个数和边数都有关
- D. 用邻接矩阵法存储图, 占用的存储空间大小只与图中边数有关, 而与结点个数无关

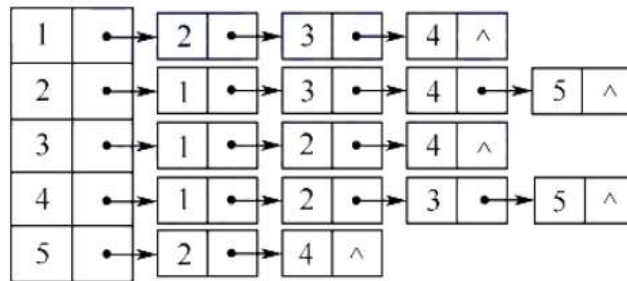
27. 下列说法错误的是 ()

- A. 对任意一个图, 从它的某个顶点出发, 进行一次深度优先或广度优先搜索, 即可访问图的每个顶点。
- B. 连通图上各边权值均不相同, 则该图的最小生成树是惟一的
- C. 有 n 个顶点的无向图, 采用邻接矩阵表示, 图中的边数等于邻接矩阵中非零元素之和的一半
- D. 树的先根遍历算法可以理解为深度优先遍历法的一种特殊形式

28. 下列说法错误的是 ()

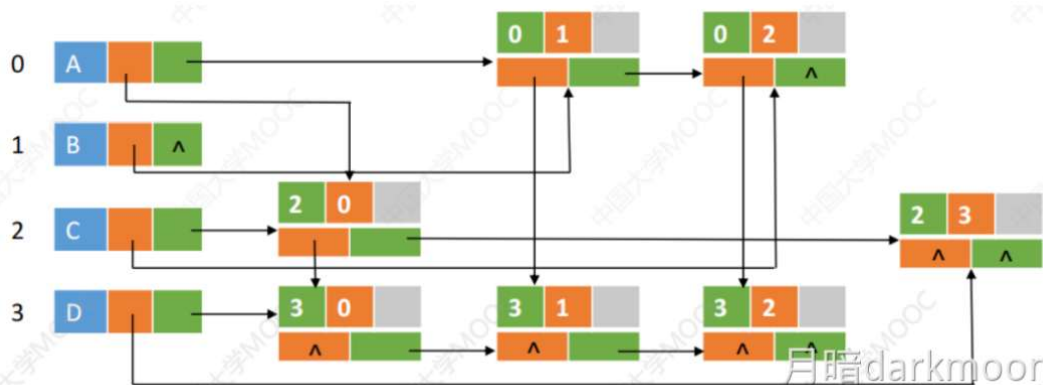
- A. 强连通图不能进行拓扑排序
- B. 只要无向网中没有权值相同的边, 其最小生成树就是唯一的。
- C. 如果表示某个图的邻接矩阵是对称矩阵, 则该图不一定是无向图。
- D. 从 n 个顶点的连通图中选取 $n-1$ 条权值最小的边, 即可构成最小生成树

29. 如下无向图的邻接表，从顶点 2 到顶点 4 长度为 2 的路径数量为 ()



- A. 1
- B. 2
- C. 3
- D. 4

30. 下图所示十字链表表示的有向图，入度为 2 的顶点有 () 个



- A. 1
- B. 2
- C. 3
- D. 4

31. 已知图的邻接矩阵，则从顶点 0 出发按深度优先遍历的结点序列是()

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

- A 0243156
- B 0136542
- C 0134256
- D 0361542

32. 下列关于广度优先算法的说法中，正确的是()。

- I.当各边的权值相等时，广度优先算法可以解决单源最短路径问题
- II.当各边的权值不等时，广度优先算法可用来解决单源最短路径问题
- III.广度优先遍历算法类似于树中的后序遍历算法
- IV.实现图的广度优先算法时，使用的数据结构是队列

- A.I、IV
- B.II、III、IV
- C.II、IV
- D.I、III、IV

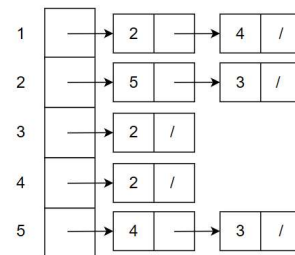
33. 下列关于深度优先遍历算法，正确的有 ()

- I. 深度优先遍历有向图过程中，DFSTraverse()调用DFS的次数是强连通分量的个数
- II. 深度优先遍历有向图过程中，回边的存在说明图中包含环
- III. 在有向图的深度优先遍历过程中，若一个顶点u访问过后，另外一个顶点v访问时发现存在到这个顶点的边<v,u>，那么该有向图一定存在环。
- IV. 深度优先遍历算法遍历有向无环图，如果在退出DFS时依次输出顶点，得到的是逆拓扑排序

- A. II、IV
- B. I、II、III
- C. II、III、IV
- D. I、II、III、IV

34. 下列邻接表表示的有向图，从顶点 1 开始进行深度优先遍历，可能得到不同遍历序列个数有 () 个

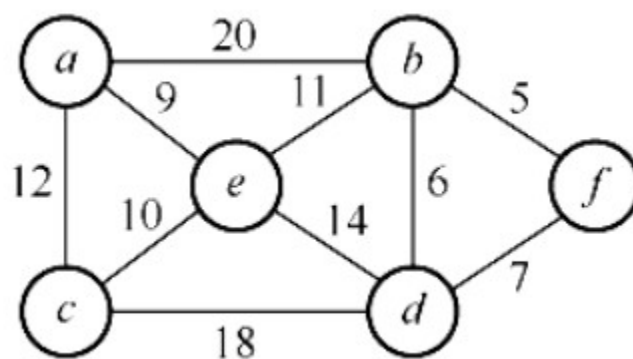
- A. 2
- B. 3
- C. 4
- D. 5



35. 用 Prim 算法求一个带权连通图的最小生成树，在算法执行的某个时刻，已选取的顶点集合 $U = \{1, 2, 3\}$ ，已选取的边集合 $TE = \{(1, 2), (2, 3)\}$ ，要选取下一条权值最小的边，应当从 () 组中选取。

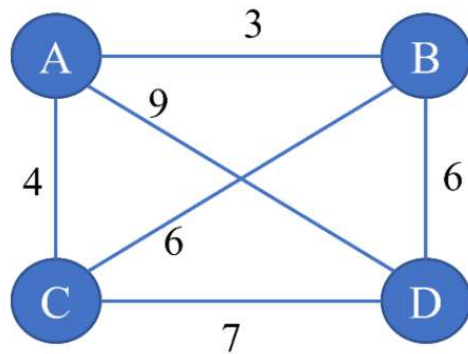
- A. $\{(1, 4), (3, 4), (3, 5), (2, 5)\}$
- B. $\{(3, 4), (3, 5), (4, 5), (1, 4)\}$
- C. $\{(1, 2), (2, 3), (3, 5)\}$
- D. $\{(4, 5), (1, 3), (3, 5)\}$

36. 已知无向图 G 如下图所示，使用 Prim 算法求图 G 的最小生成树（从顶点 b 出发），加到最小生成树中的边依次是 ()



- A. $\langle b, f \rangle, \langle b, d \rangle, \langle a, e \rangle, \langle c, e \rangle, \langle b, e \rangle$
- B. $\langle b, f \rangle, \langle b, d \rangle, \langle b, e \rangle, \langle a, e \rangle, \langle c, e \rangle$
- C. $\langle a, e \rangle, \langle b, e \rangle, \langle c, e \rangle, \langle b, d \rangle, \langle b, f \rangle$
- D. $\langle a, e \rangle, \langle c, e \rangle, \langle b, e \rangle, \langle b, f \rangle, \langle b, d \rangle$

37. 求下图的最小生成树过程中，在选择第三条边时，下列哪一条边即能是克鲁斯卡尔算法的选择也能是 Prim 算法的选择()



- A . (A, C)
- B . (A, B)
- C . (B, D)
- D . (B, C)

38. 下列关于最小生成树的陈述中，错误的是 ()

- A. 设 (u, v) 是连通图 G 中的权重最小的边，那么 (u, v) 是 G 的某棵最小生成树中的一条边
- B. 设 e 是连通图 $G=(V, E)$ 的某条环路上权重最大的边，那么图 G 中存在一棵不包含边 e 的最小生成树
- C. 若图的所有边的权重有正有负，则任一连接所有结点且总权重最小的一个边集合必然形成一棵树
- D. 若 T 是图 G 的一棵最小生成树，如果减小了 T 中的一条边的权重，那么 T 仍然是 G 的一棵最小生成树

39. 下列关于最小生成树算法 Prim 和 Kruskal 算法，正确的有 ()

- I. Prim 算法每轮选取离上一轮次选取的顶点距离最近的顶点
- II. Prim 算法的时间复杂度可优化至 $O(E \log V)$

III. 稠密图情况下，无向图采用邻接矩阵存储，**Kruskal** 算法的时间复杂度是 $O(n^2)$

IV. **Kruskal** 算法使用了并查集的数据结构

A. I、II、III、IV

B. II、III

C. II、IV

D. III、IV

40. 下列关于图的最短路径的相关叙述中，正确的是()。

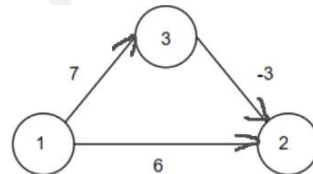
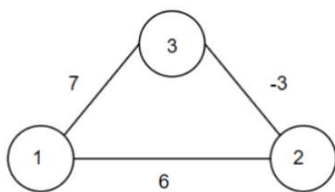
A. 最短路径一定是简单路径

B. Dijkstra 算法不适合求有回路的带权图的最短路径

C. Dijkstra 算法不适合求任意两个顶点的最短路径

D. Floyd 算法求两个顶点的最短路径时， $path_{k-1}$ 一定是 $path_k$ 的子集

41. 根据下图，关于 Dijkstra、Prim、Floyd 算法说法正确的是 ()



A. 用 Dijkstra 算法求上面右图的结点 1 单源最短路径，到结点 2 和结点 3 的最短路径长度分别为 4 和 7

B. 用 Prim 算法求左图最小生成树，从结点 1 出发，得到的最小生成树保留的边权值为 -3 和 6

C. Dijkstra 和 Prim 算法的思想都是基于贪心，都是从当前轮选出的结点作为中间结点去更新未处理结点的 $path$ 数组，且更新的策略一样

D. 用 Floyd 算法求右图的最短路径，经过一轮计算后，得到的矩阵 $A^0 = \begin{bmatrix} 0 & 4 & 7 \\ \infty & 0 & \infty \\ \infty & -3 & 0 \end{bmatrix}$

42. 下列关于图的最短路径的相关叙述中，正确的是()。

I. Dijkstra 算法求单源最短路径不允许边的权为负

II. Dijkstra 算法求每对顶点间的最短路径的时间复杂度是 $O(n^2)$

III. Floyd 算法求每对顶点间的最短路径允许边的权为负，但不允许含有负边的回路

A. I、II 和 III

B. 仅 I

C. I 和 III

D. II 和 III

43. 使用有向无环图描述表达式 $((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)$ ，需要的边的个数至少是 ()

A. 14 B. 15 C. 16 D. 13

44. 下列说法错误的是 ()

A. 求单源最短路径的 Dijkstra 算法不适用于有负权边的有向网络

B. 最短路径一定是简单路径

C. 对有向图 G ，如果以任一顶点出发进行一次深度优先或广度优先遍历能访问到每个顶点，则该图一定是完全图。

D. 拓扑排序算法不适合求无向图的拓扑序列

45. 已知有向图 $G(V, E)$ ，其中 $V=\{v_0, v_1, v_2\}$ ， $E=\{(v_0, v_1)6, (v_0, v_2)13, (v_1, v_0)10, (v_1, v_2)4, (v_2, v_0)5\}$ ，对于 Floyd 算法， A^1 是 ()

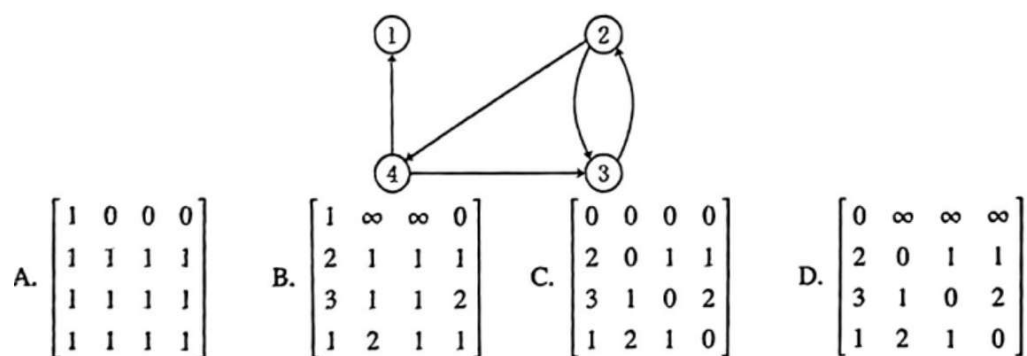
A.
$$\begin{bmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & \infty & 0 \end{bmatrix}$$

B.
$$\begin{bmatrix} 0 & 6 & 10 \\ 9 & 0 & 4 \\ 5 & 11 & 0 \end{bmatrix}$$

C.
$$\begin{bmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & 11 & 0 \end{bmatrix}$$

D.
$$\begin{bmatrix} 0 & 6 & 10 \\ 10 & 0 & 4 \\ 5 & 11 & 0 \end{bmatrix}$$

46. 用 Floyd 算法求下图的最短路径，得到的 dist 矩阵结果是 ()。



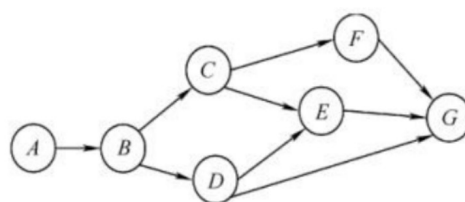
47. 下列关于最短路径算法的叙述，正确的是（）

- I. Dijkstra 算法求有向图的单源最短路径，有向图可以带负权但是不能有负权环路
- II. Dijkstra 算法的核心思想是贪心策略
- III. Floyd 算法的核心思想是动态规划
- IV. Dijkstra 算法求每对顶点之间的最短路径的时间复杂度是 $O(V^2)$

- A. I、II、III
- B. II、III、IV
- C. II、III
- D. 全部

48. 下图所示有向图的所有拓扑序列共有()个

- A.4
- B.6
- C.5
- D.7

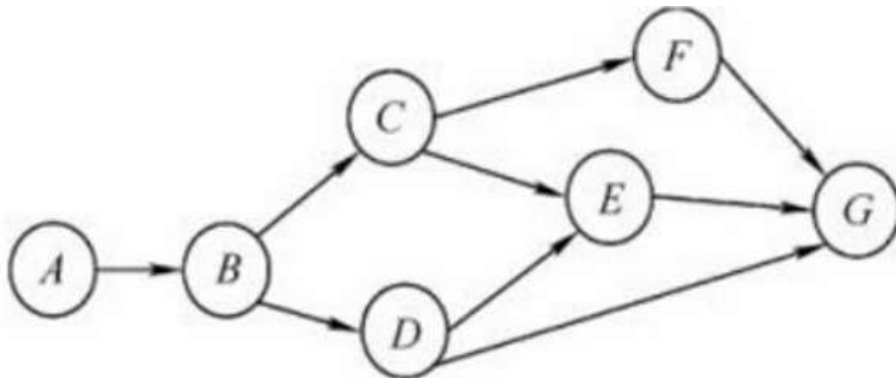


49. 已知有向图 $G=(V, E)$ ，其中 $V=\{a, b, c, d, e, f, g\}$, $E=\{<a, b>, <a, c>, <a, d>, <b, e>, <c, e>, <c, f>, <d, f>, <f, g>, <e, g>\}$ G 的拓扑序列是（）

- A a, c, d, f, b, e, g
- B a, c, b, f, d, e, g;
- C a, c, d, e, b, f, g

D a, b, e, c, d, f, g

50. 下图所示有向图的所有拓扑序列共有()个



- A.4
- B.6
- C.5
- D.7

51. 下列关于图的说法中，正确的是()。

- I.有向图中顶点 V 的度等于其邻接矩阵中第 V 行中 1 的个数
 - II.无向图的邻接矩阵一定是对称矩阵，有向图的邻接矩阵一定是非对称矩阵
 - III.在带权图 G 的最小生成树 G_s 中，某条边的权值可能会超过未选边的权值
 - IV.若有向无环图的拓扑序列唯一，则可以唯一确定该图
- A.I、II 和 III

B.III 和 IV

C.III

D.IV

52. 下列说法错误的是 ()

- A. 若一个有向图存在拓扑序列，则其邻接矩阵一定是上三角或下三角矩阵
- B. 强连通分量是有向图中的极大强连通子图
- C. 连通图的广度优先遍历中一般要采用队列来暂存刚访问过的顶点
- D. 图的深度优先遍历中一般要采用栈来暂存刚访问过的顶点

53. 在有向图 G 的拓扑序列中, 若顶点 V_i 在顶点 V_j 之前, 则下列情形不可能出现的是 ()

- A. G 中有弧 $\langle V_i, V_j \rangle$
- B. G 中有一条从 V_i 到 V_j 的路径
- C. G 中没有弧 $\langle V_i, V_j \rangle$
- D. G 中有一条从 V_j 到 V_i 的路径

54. 若一个有向图中的顶点不能排成一个拓扑序列, 则可断定该有向图 ()

- A. 是个有根有向图
- B. 是个强连通图
- C. 含有多个入度为 0 的顶点
- D. 含有顶点数目大于 1 的强连通分量

55. 下列说法正确的是 ()

- I、DFS 可实现有向无环图的拓扑排序
- II、Dijkstra 算法和 Prim 算法都是基于贪心算法, Dijkstra 算法不一定得到一棵最小生成树
- III、AOE 网使用顶点表示事件, 边表示活动
- IV、邻接矩阵为上三角或下三角的图一定能得到有序的拓扑序列

- A. II、III、IV B. I、III、IV C. I、II D. 全部

56. 下列关于 AOE 网的说法正确的是 ()。

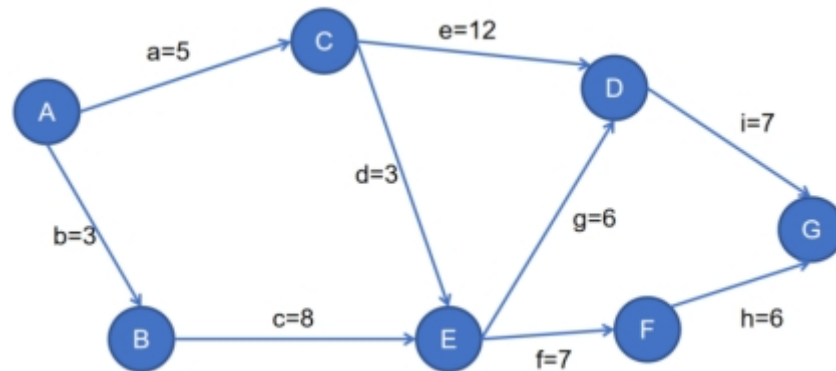
- A. AOE 网的含义是以顶点表示活动的网
- B. 从源点到汇点的最短路径称作关键路径
- C. 存在关键路径的 AOE 网一定是有向无环图
- D. 关键路径上某个活动的时间缩短, 整个工程的时间也就必定缩短。

57. 下面的叙述中, 不正确的是 ()

- A 关键活动不按期完成就会影响整个工程完成时间

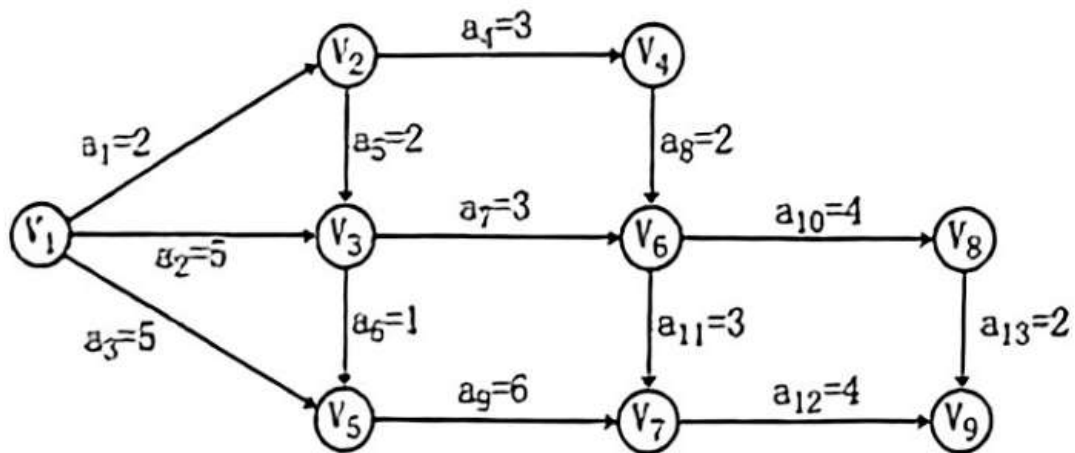
- B 任何一个关键活动提前完成，将使整个工程提前完成
- C 所有关键活动提前完成，则整个工程提前完成
- D 某些关键活动若提前完成，将可能使整个工程提前完成

58. 下图所示 AOE 网表示一项包含 8 个活动的工程，通过同时加快若干活动的进度可以缩短整个工程的工期。下列选项中加快其进度也不可以缩短工程工期的是 ()



- A. a, g, h B. b, g, i C. a, g, f D. a, d, h

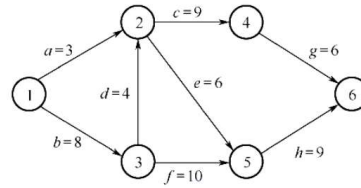
59. 下图的关键路径长度是 ()。



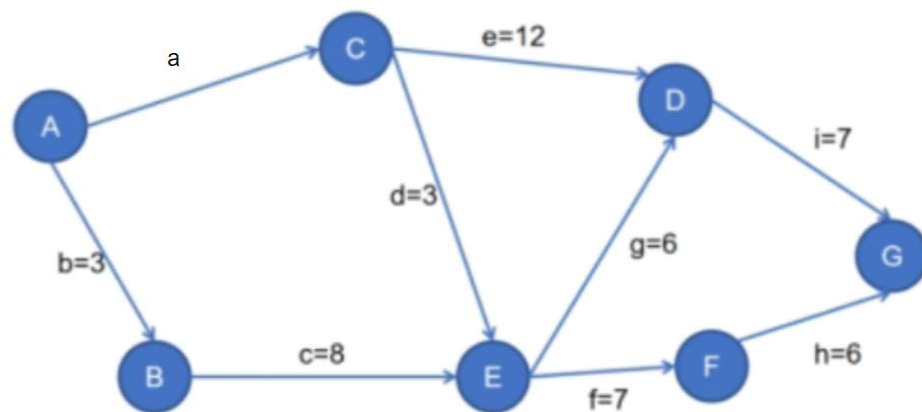
- A. 16 B. 15 C. 18 D. 20

60. 下图是一个 AOE 网，时间余量最大的活动是 ()

- A. a
- B. c
- C. e
- D. d



61. 下图所示 AOE 网表示一项包含 8 个活动的工程，增加活动 c 或者增加活动 e 的时间，工程的工期一定延长，那么活动 a 的持续时间是()



- A. 5
- B. 6
- C. 7
- D. 8

七 查找

1. 设包含 4 个数据元素的集合 S，各元素的查找概率依次为{0.35, 0.15, 0.15, 0.35}。将 S 保存在一个长度为 4 的顺序表中，保存的顺序可调整，采用顺序查找，查找成功的平均查找长度最小为（）

- A. 2.1
- B. 2.2
- C. 2.3
- D. 2.9

2. 下列关于折半查找，说法错误的有（）

- A. 折半查找的判定树一定是一颗平衡二叉树
- B. 折半查找的判定树中，所有叶子结点（非失败结点）**不一定**在同一层。
- C. 折半查找的成功和失败平均查找长度都小于树高
- D. 小数据量的情况下，可能线性查找性能更高。

3. 下列说法错误的是（）

- A. 进行折半查找的表必须是顺序存储的有序表。
- B. 折半查找所对应的判定树是一棵平衡二叉树
- C. 对二叉排序树进行中序遍历得到的结点的值的序列是一个有序序列。
- D. 在由 n 个元素组成的有序表上进行折半查找时，对任一个元素进行查找的长度都不会大于 $\log_2(n)$

4. 下列选项中不能构成折半查找中关键字比较序列的是()。

- A. 500, 200, 450, 180
- C. 180, 500, 200, 450
- B. 500, 450, 200, 180
- D. 180, 200, 500, 450

5. 含有 10 个关键字的有序序列，关于其二分查找判定树（不含外部结点）的说法错误的是（）

I. 每个分支结点的左右子树的结点总数之差始终不变（1 或-1）

II. 度为 1 的结点个数为 3

III. 是一棵完全二叉树

IV. 是一棵二叉排序树

A. I、II、IV

B. I、III

C. II、IV

D. I、II、III、IV

6. 折半查找有序表(4,6,10,12,20,30,50,70,88,100)。若查找表中元素 58，则它将依次与表中()比较大小，查找结果是失败。

A. 20、70、30、50

B. 30、88、50、70

C. 20、50

D. 30、88、50

7. 对 511 个关键字的有序序列进行折半查找，查找失败的平均查找长度为（）

A. $\frac{4097}{511}$

B. $\frac{4097}{512}$

C. 9

D. 10

8. 设查找表中有 100 个元素，如果用二分法查找方法查找数据元素 X，则最多需要比较（）次就可以断定数据元素 X 是否在查找表中

A 5

B 6

C 7

D 8

9. 假设在有序线性表 $A[1..30]$ 上进行二分查找,则比较五次查找成功的结点数为()

A.8

B.12

C.15

D.16

10. 下列关于查找的说法中, 正确的是()

A.若数据元素保持有序, 则查找时就可以采用折半查找法

B.折半查找与二叉查找树的时间性能在最坏情况下是相同的

C.折半查找法的平均查找长度一定小于顺序查找法

D.折半查找法查找一个元素大约需要 $O(\log_2 n)$ 次关键字比较

11. 对于具有 152 个记录的文件, 采用分块查找法, 且每块长度为 8。若索引表和各块内均用顺序查找, 则平均查找长度为 ()

A. 4.5

B. 9

C. 14.5

D. 13.5

12. 当采用分块查找时, 数据的组织方式为()

A. 数据分成若干块, 每块内数据有序

B. 数据分成若干块, 每块内数据不必有序, 但块间必须有序, 每块内最大(或最小)的数据组成索引块

C. 数据分成若干块, 每块内数据有序, 每块内最大(或最小)的数据组成索引块

D. 数据分成若干块, 每块(除最后一块外)中数据个数需要相同

13. 若对 961 个记录的索引顺序表进行分块查找, 块数为 31, 索引查找采用二分查找, 块内查找为顺序查找, 那么查找成功的平均查找长度为 () (结果向上取整)

A. 20

- B. 21
- C. 25
- D. 26

14. 对有 2500 个记录的索引顺序表(分块表)进行查找, 最理想的块长为()。

- A. 50
- B. 125
- C. 500
- D. $\log_2(2500)$

15. 下面是关于二叉排序树的说法, 正确的是 ()

- A. 已知二叉排序树的关键字的先序遍历序列, 可以确定该二叉排序树
- B. 对于同样一组记录, 等概率成功查找时, 二叉排序树的平均查找时间一定小于顺序表的平均查找时间
- C. n 个结点的二叉排序树有多种, 等概率成功查找时, 其中树高最小的二叉排序树不一定是平均查找长度最佳的
- D. 在任意一棵非空二叉排序树中, 删除某结点后又将其插入, 则所得二叉排序树与原二叉排序树相同

16. 分别以下列序列构造二叉排序树, 与用其它三个序列所构造的结果不同的是 ()

- A (100, 80, 60, 90, 120, 130, 110)
- B (100, 120, 110, 130, 80, 60, 90)
- C (100, 80, 90, 60, 120, 110, 130)
- D (100, 60, 80, 90, 120, 110, 139)

17. 若想查找 63, 哪个最有可能是 在二叉排序树上进行的查找 ()

- A . 2 25 101 39 80 70 59 63
- B . 39 101 25 80 70 59 63
- C . 101 70 2 39 25 59 80 63
- D . 59 2 80 70 39 25 63

18. 已知 10 个元素(54, 28, 16, 34, 73, 62, 95, 60, 26, 43), 按照依次插入的方法生成一棵二叉排序树, 查找值为 62 的结点所需比较次数为()

- A.2
- B.3
- C.4
- D.5

19. 高度为 8 的平衡二叉树的结点数至少为 ()

- A.15
- B.54
- C.87
- D.255

20. 含有 18 个结点的平衡二叉树的最大深度为()。

- A. 4
- B. 5
- C. 6
- D. 7

21. 在含有 15 个结点的平衡二叉树上, 查找关键字为 28 的叶子结点(存在该结点), 则依次比较的关键字有可能是()

- A . 30, 28
- B . 38, 48, 28
- C . 48, 18, 38, 28
- D . 60, 20, 50, 40, 38, 28

22. 在平衡二叉树中插入一个节点后造成了不平衡, 设最低的不平衡节点为 A, 并已知 A 的左孩子的平衡因子为 0, 右孩子的平衡因子为 1, 则应作()型调整以使其平衡。

- A. LL
- B. LR
- C. RL
- D. RR

23. 一棵结点数为 12 的平衡二叉树，其最大高度为 () (根节点高度为 1)；叶子结点最多为 () 个。

- A. 4, 5
- B. 5, 5
- C. 5, 6
- D. 4, 6

24. 高度为 3 的平衡二叉排序树的形态共有 () 种。

- A. 13
- B. 14
- C. 16
- D. 15

25. 5 个结点的平衡二叉树有多少种不同的形态 ()

- A. 5
- B. 6
- C. 7
- D. 8

26. 若将关键字 1,2,3,4,5,6,7 依次插入初始为空的平衡二叉树 T, 则 T 中平衡因子为 0 的分支结点的个数是 ()

- A. 0
- B. 1
- C. 2
- D. 3

27. 下图有可能是一棵刚做过 BST 的 () 操作，但尚未旋转调整的 AVL 树。

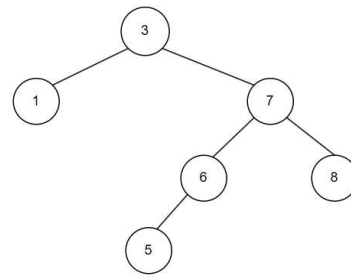
I. delete(2) II. insert(3) III.
delete(4) IV. insert(5) V. insert(8)

A. II、III、V

B. I、IV

C. II、IV

D. I、III、IV



28. 下列关于二叉排序树和平衡二叉树说法，正确的是()

A. 对一棵二叉排序树，删除其中某个结点，再插入关键字与其相同的结点，二叉排序树不变

B. 对一棵二叉排序树，插入某个结点，再删除这个插入的结点，二叉排序树不变

C. 对一棵平衡二叉树，删除其中某个结点，再插入关键字与其相同的结点，平衡二叉树不变

D. 对一棵平衡二叉树，插入某个结点，再删除这个插入的结点，平衡二叉树不变

29. 下列关于红黑树的说法正确的是 ()

I. 红黑树的某棵子树（如果有的话），也一定是红黑树。

II. 如果根结点黑高为 h ，则内部结点数（含关键字的结点）最多有 $2^{2h} - 1$ 个

III. 红结点数量不会超过内部结点数量的一半

IV. 任何一个结点的左右子树高度之差不超过 2 倍

V. 红黑树和平衡二叉树的查找、插入、删除操作，最坏时间复杂度数量级相同。

A. II、III、IV、V

B. I、IV、V

C. II、IV、V

D. III、IV、V

30. 下列关于红黑树和 AVL 树的说法中，不正确的是()。

I. 一棵含有 n 个结点的红黑树的高度至多为 $2\log_2(n+1)$

II. 若一个结点是红色的，则它的父结点和孩子结点都是黑色的

III. 红黑树的查询效率一般要优于含有相同结点数的 AVL 树

IV. 若 AVL 树的某结点的左右孩子的平衡因子都是零，则该结点的平衡因子也是零

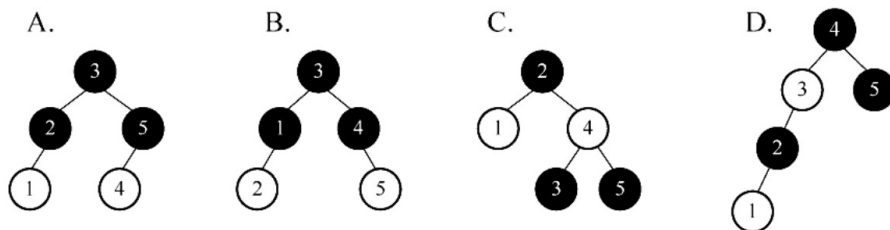
A. I、III

B. III

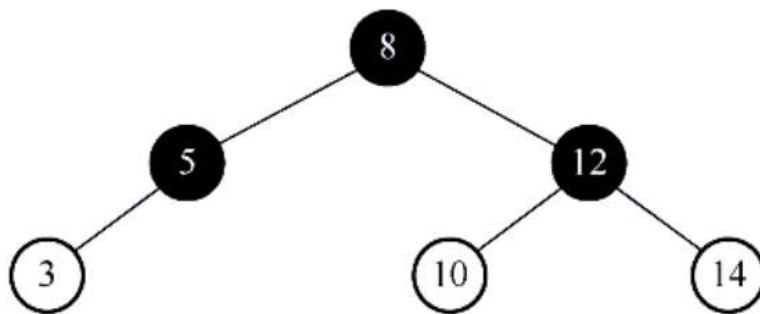
C.II、IV

D.III、IV

31. 下列满足红黑树定义的是()



32. 在下图所示的红黑树中插入结点 2 且染成红色后，则下一步应进行的操作是()。



A.左旋

B.右旋

C.变色

D.无需调整

33. 将关键字 41, 38, 31, 12, 19, 8 依次插入空的红黑树后，下列对结果的树说法错误的是 ()。

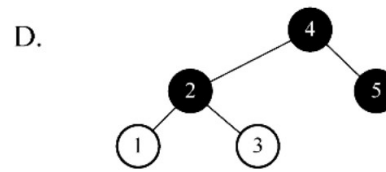
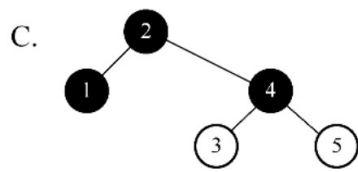
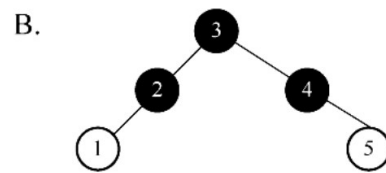
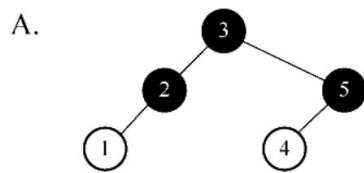
A. 有 2 个红色结点， 4 个黑色结点

B. 12 和 41 的黑高都为 0

C. 8 是 12 的孩子

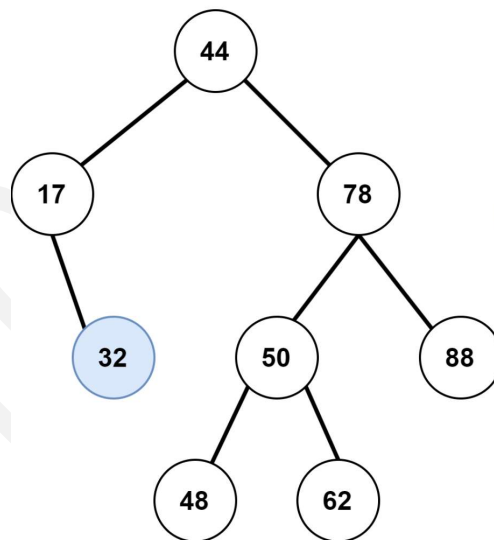
D. 树中存在相邻的黑色内部结点

34. 将关键字 5,4,3,2,1 依次插入初始为空的红黑树 T，则 T 的最终形态是()。

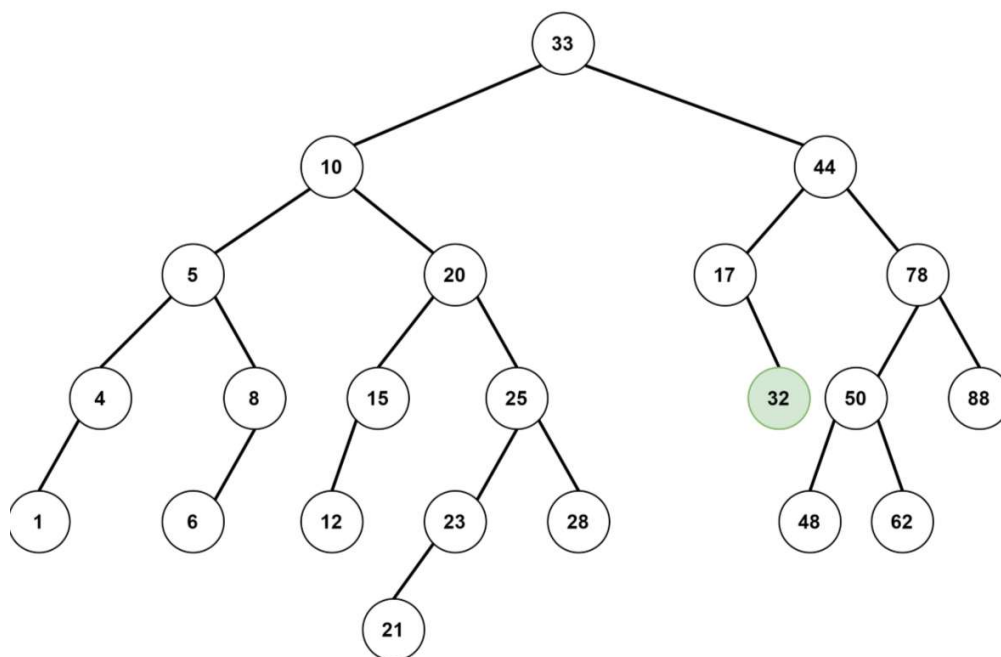


35. 右图平衡二叉树，删除结点 32，经过 () 调整

- A. LL
- B. LR
- C. RL
- D. RR



36. 下图平衡二叉树，删除结点 32，要经历()次旋转



- A. 1
- B. 2
- C. 3
- D. 4

37. 下列关于 B 树的说法错误的是 ()。

- A. 高度为 h 的 m 阶 B 树最多存储 $m^h - 1$ 个关键字
- B. 所有叶结点都在同一层次上
- C. 含 n 个结点 (不含失败结点) 的 m 阶 B 树至少包含 $(n - 1) \times ([m/2] - 1) + 1$ 个关键字
- D. 和二叉搜索树相同, B 树高度的增加也是发生在底部

38. 下列数据结构中, 从根结点到任意叶结点的路径都是有序的是 ()。

- A. AVL
- B. 二叉查找树
- C. 红黑树
- D. 堆

39. 在一棵含有 n 个关键字的 m 阶 B 树中进行查找, 至多读盘 () 次

- A. $\log_2 n$
- B. $1 + \log_2 n$

C. $\log_{\lceil m/2 \rceil} \lceil (n+1)/2 \rceil + 1$

D. $\log_{\lceil n/2 \rceil} \lceil (m+1)/2 \rceil + 1$

40. 下列关于 B 树的说法错误的是 ()

I. 查找 B 树的一个结点的前驱结点就是在找其左孩子的最右结点

II. B 树支持顺序查找

III. B 树每次都需查询到叶结点, 查询性能稳定

IV. 使用 B 树进行范围查找时, 需要依赖中序遍历

A II,III

B III

C III,IV

D II,IV

41. 初始为空的 3 阶 B 树依次插入 10, 26, 16, 12, 3, 28, 18, 插入并调整结束后, 根节点的关键字为 ()

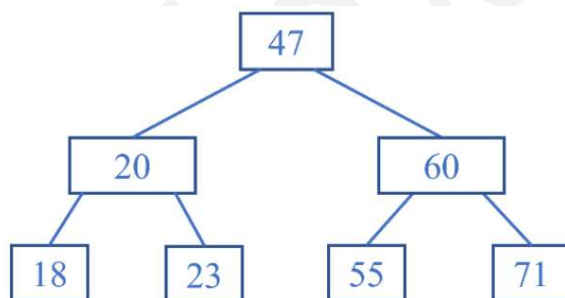
A. 10

B. 16

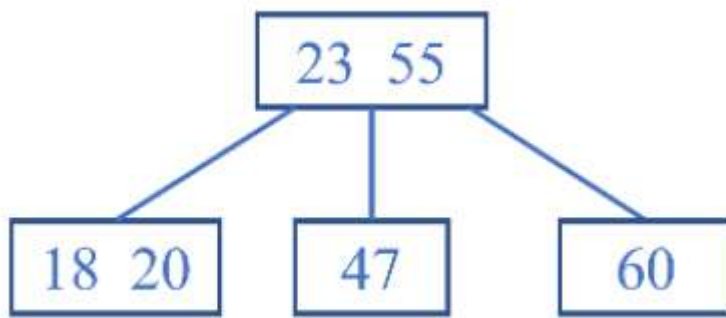
C. 10, 16

D. 10, 16, 26

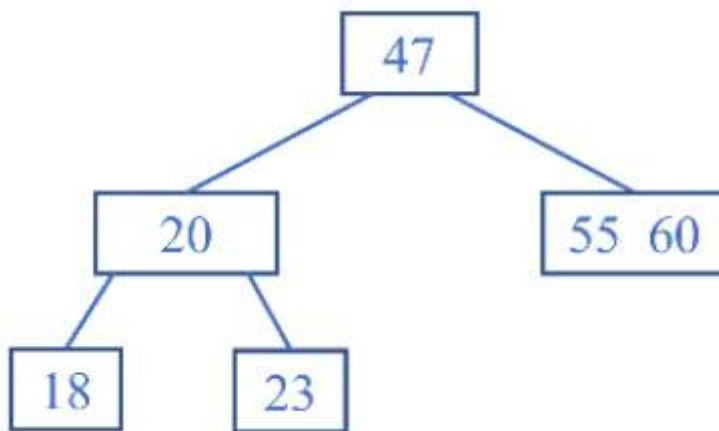
42. 对于下面这棵 3 阶 B 树, 完成 删除 71 的操作后应该是 ()



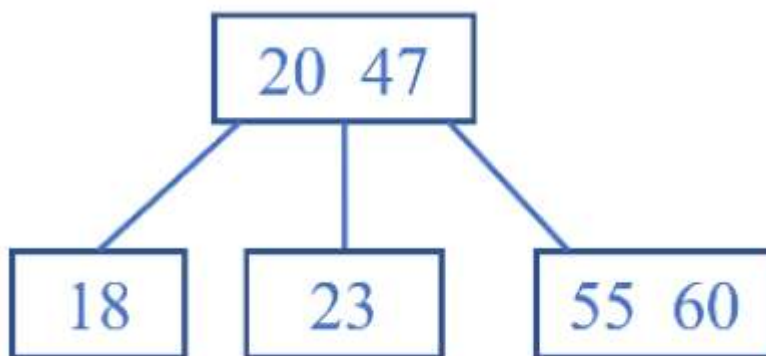
A.



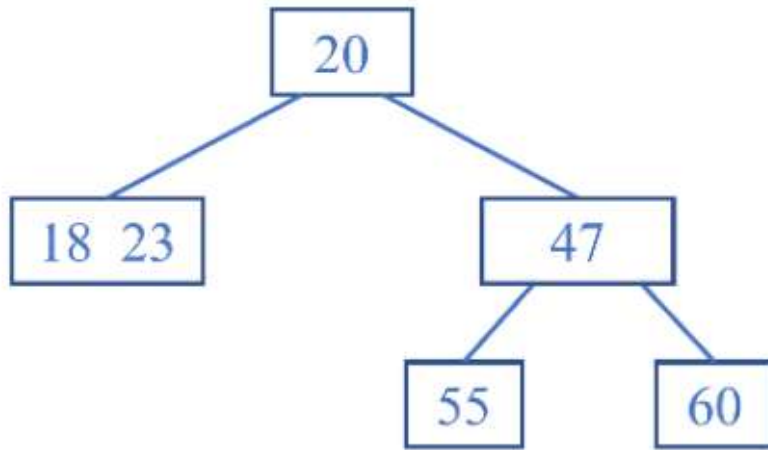
B.



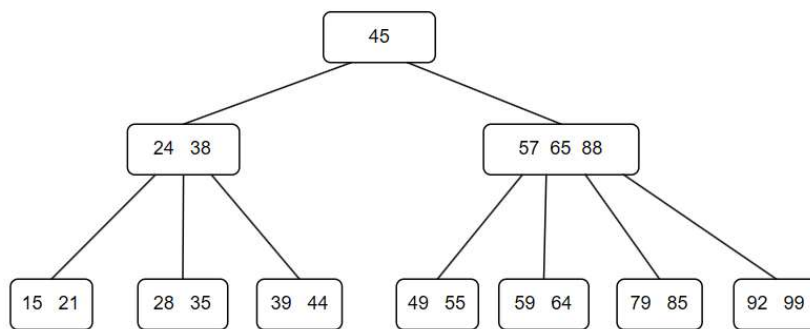
C.



D.



43. 下图所示 5 阶 B 树，删除关键字 24 后，再删除 59，所得 B 树的根节点是 ()



- A. 45
- B. 57
- C. 38, 45, 57, 88
- D. 45, 64

44. 下面关于 m 阶 B+树的叙述，正确的是()

- A. 每个结点内的关键字个数范围为 $\lceil \frac{m}{2} \rceil \sim m$
- B. 所有叶子结点（含关键字）的关键字个数和是 n，那么非叶子结点关键字个数总和也为 n
- C. 方便范围查找，也支持顺序查找
- D. B+树查找时，不一定要查到最后一层。

45. 下面关于 B 树和 B+树的叙述, 不正确的结论是()

- A.B 树和 B+树都能有效支持顺序查找
- B.B 树和 B+树都能有效支持随机查找
- C.B 树和 B+树都是平衡的多叉树
- D.B 树和 B+树都可用于文件索引结构

46. 已知一棵 5 阶 B 树中共有 53 个关键字, 则树的最大高度为(), 最小高度为()。

- A.2
- B.3
- C.4
- D.5

47. 在 7 阶 B 树中搜索第 2025 个关键字, 若根结点已读入内存, 则最多需启动()次 I/O。

- A. 4
- B. 5
- C. 6
- D. 7

48. 具有 n 个关键码的 m 阶 B 树的失败节点个数最多为()

- A. $n+1$
- B. $n-1$
- C. $n * m$
- D. $\lfloor \lceil m/2 \rceil \rfloor * n$

49. 设哈希地址空间为 $0 \sim m-1$, k 为记录的关键字, 哈希函数采用除留余数法, 即 $\text{Hash}(k)=k\%p$, 为了减少发生冲突的频率, 一般取 p 为 ()

- A m
- B 小于或等于 m 的最大质数, 并不等于阶大的那个数。
- C 大于 m 的最小质数

D 小于等于 m 的最大合数

50. 哈希表采用线性探测法处理冲突，在查找某个关键词时，查找路径上的各个关键字 ()

- A. 一定是同义词
- B. 一定不是同义词
- C. 可能是同义词
- D. 都相同

51. 下列关于哈希表的说法错误的是 ()

- A. 除留余数法是所有哈希函数中最好的。
- B. 采用链地址法解决冲突时，不存在聚集现象。
- C. 在哈希表查找中，比较操作一般也是不可避免的。
- D. 若散列表的装填因子 $\alpha < 1$ ，仍可能产生冲突

52. 哈希函数为 $H(\text{key}) = \text{key} \text{ MOD } 11$ ，表中已存入关键字分别为 7、14、37、60 和 83 的五个记录，此时哈希表的装填因子 $\alpha = 0.33$ 。用二次探测再散列法解决冲突，则再放入关键字为 49 的记录时，它的位置下标是 ()

- A. 5
- B. 8
- C. 9
- D. 1

53. 已知关键字序列 {2, 52, 66, 30, 41, 21, 19, 6, 10, 16, 101, 1}，Hash 函数为 $\text{key} \% 11$ ，表长为 12，冲突解决方法为拉链法，请问该表的查找失败的 ASL 为 ()

- A. 22/11
- B. 23/11
- C. 23/12
- D. 22/12

54. 将关键字序列 (34, 69, 9, 56, 24, 44) 散列存储到散列表中，散列表的是一个一维数组 $A[0 \dots 8]$ ，散列函数 $H(\text{key}) = \text{key} \bmod 7$ ，采用线性探测法解决冲突，现删除 24，插入关键字 51，则 51 在散列表中的下标为 ()，查找失败的平均查找长度

ASL 为 ()

A. 3, 16/7

B. 5, 20/7

C. 3, 19/9

D. 5, 31/9

55. 下列关于散列冲突的说法中, 正确的是()。

A.再散列在处理冲突时不会产生“堆积”现象

B.在哈希表中进行查找是不需要进行关键字比较的

C.散列函数选的好可以减少冲突现象

D.采用线性探测法处理冲突时, 所有同义词在散列表中一定相邻

56. 下述有关 hash 冲突时候的解决方法的说法, 错误的是 ()

A 通常有两类方法处理冲突:开放定址(OpenAddressing)法和拉链(Chaining)法

B 开放定址更适合于造表前无法确定表长的情况

C 在用拉链法构造的散列表中, 删除结点的操作易于实现

D 拉链法的缺点是:指针需要额外的空间, 故当结点规模较小时, 开放定址法较为节省空间

57. 下面关于散列查找的说法, 不正确的是()

A.采用链地址法处理冲突时, 查找任何一个元素的时间都相同

B.采用链地址法处理冲突时, 若规定插入总是在链首, 则插入任一个元素的时间是相同的

C.用链地址法处理冲突, 不会引起二次聚集现象

D.用链地址法处理冲突, 适合表长不确定的情况

58. 假定有 k 个关键字互为同义词, 若用线性探测法把这 k 个关键字存入足够长的散列表中, 至少要进行 () 次探测。

A. k - 1

B. k

C. $k(k - 1) / 2$

D. $k(k + 1) / 2$

59. 如果要求一个线性表既能较快地查找，又能适应动态变化的要求，最好采用()查找法。

A. 顺序查找

B. 折半查找

C. 分块查找

D. 哈希查找

60. 散列表查找过程和给定值进行比较的关键字个数取决于()

A. 散列函数

B. 处理冲突的方法

C. 装填因子

D. 以上全部

八 排序

1. 对 n 个不同的元素进行直接插入排序, 比较次数最多为()次
 - A. n
 - B. $n-1$
 - C. $n(n+1)/2$
 - D. $n(n-1)/2$
2. 下列关于插入排序的说法正确的是 ()
 - A. 直接插入排序的平均时间复杂度是 $O(n^2)$
 - B. 折半插入排序的平均时间复杂度是 $O(n\log_2 n)$
 - C. 希尔排序是稳定的排序算法
 - D. 初始序列不同, 折半插入排序的总比较次数一定是相同的
3. 序列{20,26,40,8,6,66}只能是()两趟排序后的结果
 - A. 简单选择排序
 - B. 冒泡排序
 - C. 直接插入排序
 - D. 堆排序
4. 用直接插入排序算法对下列 4 个表进行(从小到大)排序, 比较次数最少的是()。
 - A. 94,32,40,90,80,46,21,69
 - B. 21,32,46,40,80,69,90,94
 - C. 32,40,21,46,69,94,90,80
 - D. 90,69,80,46,21,32,94,40
5. 设一组初始记录关键字序列为(50, 40, 95, 20, 15, 70, 60, 45), 则以增量 $d=4$ 的一趟希尔排序结束后前 4 条记录关键字为 ()
 - A 40, 50, 20, 95
 - B 15, 40, 60, 20

C 15, 20, 40, 45

D 45, 40, 15, 20

6. 对序列{15,9,7,8,20,-1,4}采用希尔排序, 经一趟后序列变为{15,-1,4,8,20,9,7}, 则该次采用的增量是()。

A. 1

B. 4

C. 3

D. 2

7. 下列能说明希尔排序(增量d 每轮取值依次为3、1)是不稳定的算法的初始数据序列是()

A. [49, 38, 65, 49, 76, 13]

B. [49, 38, 49, 65, 13, 76]

C. [49, 49, 13, 65, 76, 38]

D. [49, 13, 65, 38, 76, 49]

8. 对n 个不同的元素利用冒泡法从小到大排序, 在()情况下元素交换的次数最多

A.从大到小排列好的

B.从小到大排列好的

C.元素无序

D.元素基本有序

9. 若用冒泡排序方法对序列(12, 24, 36, 48, 60, 72)从大到小排序, 需要的比较次数是()

A. 5 B.10 C.15 D.25

10. 对数据序列 {4, 9, 10, 8, 5, 6, 20, 1, 2} 进行升序冒泡排序, 则下列说法正确的是 ()

- A. 至少需要 6 趟
- B. 元素的移动次数是 21 次
- C. 从后往前进行冒泡排序，第三趟的结果是 {1, 2, 4, 9, 10, 8, 5, 6, 20}
- D. 无论从前往后还是从后往前进行冒泡排序，元素的移动次数都是 22 次

11. 双向起泡排序过程可以理解为双向进行的冒泡排序，其排序过程描述如下：每一趟按照冒泡排序的方法，找到序列最大值最小值，并将其放在序列最右端/最左端。第一趟从左往右，将最大值放在序列最右端；第二趟从右往左，将最小值放在序列最左端。下一趟缩小范围重复此过程。直到可以判断序列有序为止。对数组 [4,7,8,3,5,6,10,9,1,2] 进行双向起泡排序，排序趟数为（ ）。

- A. 7
- B. 6
- C. 8
- D. 9

12. 下列关于快速排序的说法中，正确的是（ ）

- I. 快速排序算法的最坏时间复杂度为 $O(n\log_2 n)$
- II. 快速排序的空间复杂度平均为 $O(1)$
- III. 快速排序算法的性能关键在于每次划分操作均分的程度
- IV. 快速排序是一个不稳定的排序算法
- V. 快速排序每趟排序都将大数据区间分为左右小数据区间（如果有的话），小区间之间相对有序

- A. I, II, V
- B. III, IV, V
- C. II、III、IV
- D. I、IV、V

13. 能说明快速排序是不稳定的排序方法的一组关键字序列是（ ）

- A (10, 20, 30, 40, 50)

B (50, 40, 30, 20, 10)

C (20, 20, 30, 10, 40)

D (20, 40, 30, 30, 10)

14. 使用快速排序算法对数据进行升序排序，若经过一次划分后得到的数据序列是{68,11,70,23,80,77,48,81,93,88}，则该次划分的枢轴是()。

A. 11 B.70 C.80 D. 81

15. 对下列关键字序列利用快速排序进行排序时，速度最快（比较次数最少）的情形是 ()

A. 21,25,5,17,9,23,30

B. 25,23,30,17,21,5,9

C. 21,9,17,30,25,23,5

D. 5,9,17,21,23,25,30

16. 对下列关键字序列利用快速排序进行从小到大排序时，递归深度最小的序列是 ()

A. [4, 2, 6, 1, 5, 3, 8, 7]

B. [5, 2, 6, 1, 3, 4, 8, 7]

C. [7, 1, 2, 3, 4, 5, 6, 8]

D. [1, 2, 3, 4, 5, 6, 7, 8]

17. 对下列 4 个序列，以第一个关键字为基准用快速排序算法进行排序，在第一趟过程中移动元素次数最多的是()。

A.92,96,88,42,30,35,110,100

B.92,96,100,110,42,35,30,88

C.100,96,92,35,30,110,88,42

D.42,30,35,92,100,96,88,110

18. 排序过程中，对尚未确定最终位置的所有元素进行一遍处理称为一趟。下列序列中，不可能是快速排序第二趟结果的是()。

- A. 5,2,16,12,28,60,32,72
- B. 2,16,5,28,12,60,32,72
- C. 2,12,16,5,28,32,72,60
- D. 5,2,12,28,16,32,72,60

19. 下列关于简单选择排序的说法，正确的是()

- A. 移动的时间复杂度是 $O(n^2)$
- B. 每趟会确定一个元素的最终位置
- C. 比较次数最少是 $O(n)$
- D. 是稳定排序算法

20. 下列初始数据序列中，不能说明简单选择排序不稳定的是()

- A. [4, 3, 5, 7, 6, 5, 2]
- B. [2, 5, 4, 6, 5, 3, 7]
- C. [1, 6, 3, 6, 2, 6, 7]
- D. [2, 1, 3, 4, 5, 5, 7]

21. 有一组数据{12, 15, 1, 18, 2, 35, 30, 11}，用简单选择排序由小到大排序，第2趟交换数据后数据的顺序是()

- A. 11, 1, 2, 12, 35, 18, 30, 15
- B. 1, 2, 12, 18, 15, 35, 30, 11
- C. 1, 2, 11, 12, 15, 18, 30, 35
- D. 1, 2, 11, 12, 15, 18, 35, 30

22. 下列关键字序列为堆的是 ()

- A 100, 60, 70, 50, 32, 65

B 60, 70, 65, 50, 32, 100

C 65, 100, 70, 32, 50, 60

D 70, 65, 100, 32, 50, 60

23. 现有关键字序列{5,2,34,16,20,19,1}, 要求对其进行降序排列, 下列哪个选项是第二次调整堆之后的结果()

A、2,16,5,34,20,19,1

B、19,16,5,34,20,2,1

C、5,16,19,34,20,2,1

D、16,19,5,34,20,2,1

24. 使用堆排序方法排序(45, 78, 57, 25, 41, 89), 初始堆为 ()

A. 78, 45, 57, 25, 41, 89

B. 89, 78, 57, 25, 41, 45

C. 89, 78, 25, 45, 41, 57

D. 89, 45, 78, 41, 57, 25

25. 将关键字 7,5,1,4,6,8,9 依次插入初始为空的小根堆 H,得到的 H 是()。

A.1,4,5,7,6,8,9

B.1,4,7,5,6,8,9

C.1,4,5,6,7,8,9

D.1,5,4,6,8,9,7

26. 对关键字序列{23, 17, 72, 60, 25, 8, 68, 71, 52}进行堆排序, 输出两个最小关键字后的剩余堆是()

A . {23, 72, 60, 25, 68, 71, 52}

B . {23, 25, 52, 60, 71, 72, 68}

C . {71, 25, 23, 52, 60, 72, 68}

D . {23, 25, 68, 52, 60, 72, 71}

27. 堆排序分为两个阶段，其中第一阶段将给定的序列建成一个堆，第二阶段逐次输出堆顶元素。设给定序列{48, 62, 35, 77, 55, 14, 35, 98}，若在堆排序的第一阶段将该序列建成一个堆(大根堆)，那么交换元素的次数为()

A . 5 B . 6 C . 7 D . 8

28. 对数据序列{32,40,21,46,69,94,90,80}构建成为一个大根堆后，插入元素 66，则调整后，元素 66 在第 () 层（根节点为第一层）

A. 1
B. 2
C. 3
D. 4

29. 对数据序列{32,40,21,46,69,94,90,80}构建成为一个小根堆后，删除元素 21，则调整后，堆顶元素的左右孩子为()

A. 40, 32
B. 80, 40
C. 40, 80
D. 40, 46

30. 下列关于堆排序的说法，错误的是()

A. 可以将堆视为一棵完全二叉树
B. 可以将堆视为一棵二叉排序树
C. 若要升序排序，最好使用大根堆
D. 堆中根节点到任意一个叶子结点的序列都是有序的

31. 数据表 A 中有 10000 个元素，如果仅要求找出其中最大的 10 个元素，则采用方法最节省时间 ()

A.堆排序

- B.希尔排序
- C.快速排序
- D.基数排序

32. 若只进行 3 趟排序就完成对 64 个元素的多路归并排序, 则选取的归并路数最少是()。

- A. 2
- B. 3
- C. 4
- D. 5

33. 一组记录的关键字为{25, 48, 16, 35, 79, 82, 23, 40, 36, 72}, 其中分为 5 个长度为 2 的有序表, 按归并排序的方法对该序列再进行一趟归并后的结果为()

- A. 16, 25, 35, 48, 23, 40, 79, 82, 36, 72
- B. 16, 25, 35, 48, 79, 82, 23, 36, 40, 72
- C. 16, 25, 48, 35, 79, 82, 23, 36, 40, 72
- D. 16, 25, 35, 48, 79, 23, 36, 40, 72, 82

34. 已知进行 2-路归并排序的待排序记录的关键字当前序列为{49, 38, 65, 97, 76, 13, 27}, , 下列说法错误的是 ()

- A. 该序列只可能是初始状态
- B. 要进行 3 趟归并
- C. 第一趟关键字比较的次数是 4 次
- D. 第二趟归并后, 13 的相邻元素是 97, 27

35. 对{05, 46, 13, 55, 94, 17, 42}进行基数排序, 一趟排序的结果是()。

- A. 05, 46, 13, 55, 94, 17, 42
- B. 05, 13, 17, 42, 46, 55, 94
- C. 42, 13, 94, 05, 55, 46, 17
- D. 05, 13, 46, 55, 17, 42, 94

36. 对 n 个 r 进制的关键字，关键字最多有 m 位进行基数排序，时间复杂度为()

- A. $O(n)$
- B. $O(n(r + m))$
- C. $O(m(n + r))$
- D. $O(mn)$

37. 有 n 个十进制整数进行基数排序，其中最大的整数为 5 位，则基数排序过程中临时建立的队列个数是()。

- A. n
- B. 2
- C. 5
- D. 10

38. 在最后一趟排序开始之前，所有记录有可能都不在其最终位置上的是 ()

- A. 直接插入排序
- B. 冒泡排序
- C. 堆排序
- D. 快速排序

39. 在快速排序、冒泡排序、堆排序、归并排序、插入排序这五种排序算法中，属于稳定排序的有几个 ()

- A 2
- B 3
- C 4
- D 5

40. 下列说法错误的是()

A 只有在线性表的初始状态为反序的情况下，冒泡排序过程中元素的移动次数才会达到最大值

B 只有在线性表的初始状态为反序的情况下，简单选择排序过程中元素的移动次数才会达到最大值

C 只有在线性表的初始状态为反序的情况下，在直接插入排序过程中元素的移动次数才会达到最大值

D 对 n 个元素进行快速排序，在进行第一次划分时，关键字的比较次数总是 $n-1$ 次

答案：B。【解析】例如对于序列[4, 3, 2, 1]（反序）和[3, 4, 2, 1]（非反序），使用简单选择排序，前者需要交换元素 2 次，后者需要交换元素 3 次。

41. 下列()算法平均情况、最好情况和最坏情况下的时间复杂度都为 $O(n^2)$

A 直接选择排序

B 直接插入排序

C 冒泡排序

D 归并排序

42. 下列关于排序和查找，说法错误的有（）

I. 若借助二分查找确定每个元素的插入位置，序列的插入排序只需 $O(n\log n)$ 时间。

II. 对任意基于比较的排序算法，最坏情况下的时间复杂度不低于 $O(n\log n)$

III. 相对有序的序列适合使用插入排序作为排序算法。

IV. 对于同一有序序列，每次折半查找绝不会慢于顺序查找

A. I、IV B. I、II、IV C I、III、IV D. I、II、III、IV

43. 下列排序算法中，某一趟排序结束后未必能选出一个元素放在其最终位置上的是（）

A. 直接插入排序

B. 冒泡排序

C. 快速排序

D. 堆排序

44. 以下排序方式中平均空间复杂度是 $O(n)$ 的是 ()

- A 简单选择排序
- B 快速排序
- C 堆排序
- D 二路归并排序

45. 以下 () 算法的最坏时间复杂度可以达到 $O(n\log n)$

- A.归并排序
- B.快速排序
- C.冒泡排序
- D.插入排序

46. 设线性表中每个元素有两个数据项 k_1 和 k_2 , 现对线性表按以下规则进行排序:先看数据项 k_1 , k_1 值小的元素在前, 大的元素在后;在 k_1 值相同的情况下, 再看 k_2 , k_2 值小的元素在前, 大的元素在后。满足这种要求的排序算法是()

- A.先按 k_1 进行直接插入排序, 再按 k_2 进行简单选择排序
- B.先按 k_2 进行直接插入排序, 再按 k_1 进行简单选择排序
- C.先按 k_1 进行简单选择排序, 再按 k_2 进行直接插入排序
- D.先按 k_2 进行简单选择排序, 再按 k_1 进行直接插入排序

47. 选择一个排序算法时, 除算法的时空效率外, 下列因素中, 还需要考虑的是()

I.数据的规模 II. 数据的存储方式 III.算法的稳定性 IV.数据的初始状态

- A. 仅 III
- B. 仅 I、II
- C. 仅 II、III、IV
- D. I、II、III、IV

48. 下列排序方法中, 时间性能与待排序记录的初始状态无关的是()

- A . 插入排序和快速排序

- B. 归并排序和快速排序
- C. 选择排序和归并排序
- D. 插入排序和归并排序

49. 对各种内部排序方法来说 ()。

- A. 快速排序时间性能最佳
- B. 基数排序和归并排序是稳定的排序方法
- C. 快速排序是一种选择排序
- D. 堆排序所用的辅助空间较大

50. 一台计算机具有多核 CPU, 可以同时执行相互独立的任务。归并排序的各个归并段可以并行执行, 在下列排序算法中, 不可以并行执行的有()。

- I. 基数排序
 - II. 冒泡排序
 - III. 快速排序
 - IV. 堆排序
- A. I、III
 - B. I、II
 - C. I、III、IV
 - D. II、IV

51. 对大部分元素已有序的数组进行排序时, 选择冒泡排序而不选择简单选择排序, 其原因是()。

- I. 冒泡排序过程中元素之间的比较次数更少
 - II. 冒泡排序过程中所需要的辅助空间更少
 - III. 冒泡排序是稳定的
- A. I
 - B. III
 - C. I, II

D. I, III

52. 以下排序算法中, ()不需要进行关键字的比较。

- A.快速排序
- B.归并排序
- C.基数排序
- D.堆排序

53. 在下列排序算法中, 平均情况下空间复杂度为 $O(n)$ 的是(), 最坏情况下空间复杂度为 $O(n)$ 的是()。

- I. 希尔排序 II.堆排序 III.冒泡排序
- IV.归并排序 V.快速排序 VI.基数排序
- A. I、IV、VI
- B. II、V
- C. IV、V
- D. IV

54. 以下关于外部排序的说法错误的是 ()

- A.影响外部排序的时间因素主要是内存与外存交换信息的总次数
- B.外部排序的两个基本操作过程是生成初始归并段和归并
- C.采用败者树进行 k 路平衡归并, 其总的归并效率与 k 无关
- D.外部排序通过减少初始归并段个数和增加归并路数可以提高排序效率

55. 在做 m 路平衡归并排序的过程中,为实现输入/内部归并/输出的并行处理,需要设置(①)个输入缓冲区和(②)个输出缓冲区。

- ① A. 2 B. m C. $2m-1$ D. $2m$
- ② A. 2 B. m C. $2m-1$ D. $2m$

56. 若外部存储上有 3110400 个记录, 做 6 路平衡归并排序, 计算机内存工作区能容纳 400 个记录, 则排序好所有记录, 需要作几趟归并排序 ()

A 6

- B 3
- C 5
- D 4

57. 假设文件有 2000 个记录，磁盘块每块可容纳 125 个记录，用于内部排序的内存工作区容量为 250 个记录，采用二路平衡归并排序对文件进行排序。那么文件从初始无序的状态到最后有序的状态，总的 I/O 次数为 ()

- A. 32
- B. 96
- C. 128
- D. 64

58. m 个初始归并段采用 k 路平衡归并时，构建的败者树中共有 () 个节点 (不计冠军节点)

- A、 $2m-1$ B、 $2m$ C、 $2k$ D、 $2k-1$

59. 在使用败者树进行外部排序时，败者树有以下 () 特点。

- A. 非叶子结点存储的是败者的标号
- B. 叶子结点存储的是输入数据
- C. 败者树只适用于内部排序
- D. 败者树的根结点存储的是最小值

60. 下列关于败者树的说法错误的有 ()

- I. 败者树的分支结点存储的内容是失败 (较大) 记录关键字的值
- II. 败者树每次维护，都必须从叶子结点一直比较到根节点 (直到选出冠军节点)
- III. 使用败者树从 n 个归并段中选出最小值，空间上的开销是 $O(\log_2 n)$
- IV. 败者树为了防止某个归并段记录被选完影响其他缓冲区记录的进一步排序，可以在每个归并段的最后添加一个关键字为最大的记录，当冠军节点为“最大值”时，表示归并结束。

- A. I、IV
- B. I、III
- C. II、IV
- D. II、III

61. 现有五个归并段，每个归并段(0-4 编号)的所有记录关键字如下：{10, 15, 16}、{9, 18, 20}、{20, 22, 40}、{6, 15, 25}、{12, 37, 48}。若采用败者树实现 5 路归并，在进行第二轮比较后（得到第二个冠军节点），根节点的值为（）

- A. 1
- B. 0
- C. 9
- D. 10

62. 设由置换-选择排序法得到 9 个初始归并段，其长度(记录个数)依次为(9,30,12,18,3,17,2,6,24)。则由其构成的 3 路最佳归并树的 I/O 次数为()

- A.440
- B.442
- C.444
- D.446

63. 对于输入数据流(15,19,04,83,12,27,11,25,16,34,26,07,10,90,06),在使用置换-选择排序算法进行外部排序时，设工作区大小为 4，那么在置换-选择排序过程中，生成的初始归并段的数量是()。

- A.2
- B.3
- C.4
- D.5

64. 下面数据结构是选择置换排序内存工作区的存储结构定义：

```
C++
typedef struct
{
    RedType rec;    //记录
```

```
KeyType key;    //从记录中抽取的关键字
int rnum;       //所属归并段的段号
RcdNode WorkArea[w];NorkArea wa;    //内存工作区，容量为 w
```

使用败者树来选出每一轮的 MINIMAX 记录。相比于普通的败者树，选择置换排序中的败者树在叶子节点，除了记录的关键字（索引）外，还引入了段号，规则为：

- 败者树的建立时，所有叶子节点初始关键字值为 0 段号为 0
- 依次从后往前将待排序序列记录的关键字输入到叶子结点，段号置为 1。至编号最小的叶子结点被填充完，第一个冠军节点（MINIMAX 记录的编号）被选出。
- 后续冠军节点所在叶子结点补充新的记录的关键字后，若小于上一轮的 MINIMAX，段号+1。否则仍为当前轮次的段号。比较的规则如下：
 - 先比较段号，段号小者直接获胜。
 - 段号相等，记录的关键字值小的获胜。

根据上述规则，对{51, 49, 39, 46, 38, 29, 14, 61, 15, 30, 48}进行选择置换排序，工作区 WA 的大小 $w=6$ ，得到第一个初始归并段后（即新的冠军结点内的编号所在叶子结点的段号是 2 的时候），败者树的根节点编号为（）

- A. 0
- B. 1
- C. 3
- D. 4

65. 在使用置换-选择排序算法进行外部排序时，以下哪个选项描述了算法如何处理候选区和输入数据流的关系?()

- A.从候选区中选择最大记录，并在输入数据流中找到一个比其大的记录进行替换
- B.从候选区中选择最小记录，并在输入数据流中找到一个比其小的记录进行替换
- C.从输入数据流中选择最小记录，并与候选区中的最大记录进行交换
- D.将输入数据流中当前读取的记录插入到候选区中适当的位置，然后选择候选区中的最小记录输出

66. 用败者树筛选 MINIMAX 记录的选择置换排序算法中，若内存工作区 WA 大小为 w 个记录，共有 n 个记录，生成所有的初始归并段所需的时间复杂度为（）

- A. $O(n)$
- B. $O(mn)$

C. $O(m\log_2 n)$

D. $O(n\log_2 m)$

答案：D。【解析】败者树每次选择一个 MINMAX 记录的时间复杂度是 $O(\log_2 m)$ ，一共要选择 $n-1$ 轮，所以整体的时间复杂度是 $O(n\log_2 m)$ 。本题细节应结合上一题理解。

67. 下列关于选择 - 置换排序的说法正确的是 ()。

I. 创建初始段文件过程中，需要不断地在内存工作区中选择不小于旧的 MINMAX 的最小值，此过程需利用败者树实现

II. 利用败者树不断选择新的 MINMAX 记录时，为防止新加入的关键字值更小，每个叶结点都附加一个段号，当进行关键字比较时，先比较段号，段号大的为胜者；段号相同的关键字值小的为胜者

III. 置换选择排序算法得到的初始归并段的长度并不受内存容量的限制，且获得的归并段的平均长度为内存工作区大小的两倍

A. I, III

B. II, III

C. I, II, III

D. I, II

68. 外部排序的过程中开销主要是 I/O 操作，那么可以采取一些措施来减小 I/O 时间，下列说法正确的是 ()

A. 使用置换-选择排序主要是为了通过减小初始归并段的长度来减少元素之间比较次数的

B. 使用败者树可以对置换-选择排序做优化来减小元素之间的比较次数

C. 败者树主要是为了增大归并路数，只要这棵败者树的路数越多，排序时间就越短

D. 构建的最佳归并树和哈夫曼树一样，只有度为 0 和 2 的结点

69. 设外存上有 80 个归并段，进行 8 路平衡归并，为实现最佳归并，需要补充的虚段个数是 ()

A. 4

B. 5

C. 6

D. 不需要补充

70. 在对大量数据进行外部排序时，以下策略不能有效降低磁盘 I/O 操作次数的是
()

- A. 选择置换排序
- B. 败者树
- C. 最佳归并树
- D. 适当增大归并路数